

第四章 TCP/IP协议栈（面试突击★★★★★）

阅读指南: 本章是后台开发面试中最核心的网络部分，TCP 的三次握手、四次挥手、拥塞控制、TIME_WAIT 是必考题。建议边画图边背诵。

目录

- 1. [OSI 7层模型 vs TCP/IP 4层模型](#)
- 2. [TCP Deep Dive](#)
 - [TCP头部格式](#)
 - [三次握手](#)
 - [四次挥手](#)
 - [TIME_WAIT](#)
 - [CLOSE_WAIT](#)
 - [滑动窗口](#)
 - [流量控制](#)
 - [拥塞控制](#)
 - [Nagle算法](#)
 - [粘包拆包](#)
- 3. [UDP](#)
- 4. [IP](#)
- 5. [ARP](#)
- 6. [DNS](#)
- 7. [HTTP/HTTPS](#)
- 8. [Socket基础](#)
- 9. [面试重点汇总](#)

1. OSI 7层模型 vs TCP/IP 4层模型

1.1 对比表

层	OSI 7层	TCP/IP 4层	功能 / 协议
7	应用层 (Application)	应用层 (Application)	为应用程序提供网络服务 HTTP, FTP, SMTP, DNS, SSH, Telnet, SNMP
6	表示层 (Presentation)		数据格式转换、加密解密 JPEG, SSL/TLS, ASCII

5	会话层 (Session)		建立/管理/终止会话 RPC, NetBIOS
4	传输层 (Transport)	传输层 (Transport)	端到端通信 TCP (可靠), UDP (尽力) 端口号 (识别进程)
3	网络层 (Network)	网络层 (Internet)	路由选择、逻辑寻址 IP, ICMP, ARP, IGMP IP地址 (识别主机)
2	数据链路层 (Data Link)	网络接口层 (Network Interface)	相邻节点间帧传输 MAC地址 (识别网卡) Ethernet, WiFi, PPP
1	物理层 (Physical)		比特流传输 光纤、双绞线、无线电

为什么 TCP/IP 要将三层合并为一层？

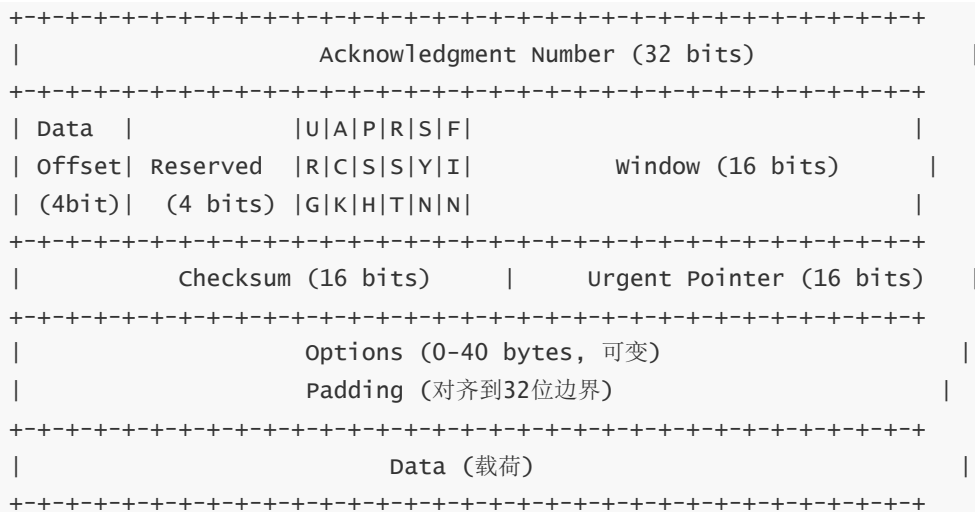
TCP/IP 模型是**先有协议后有模型**的。在设计 TCP/IP 协议栈时，表示层和会话层的功能被融入到应用层中 (如 TLS/SSL 处理加密，HTTP 处理会话)，没有单独抽象出来。这种"实用主义"的设计降低了复杂度。

1.2 各层职责详解

层	核心问题	数据单元	地址	设备	关键功能
应用层	做什么? (What)	消息/数据	域名/URI	网关/代理	为应用提供服务
传输层	发给谁? (Which Process)	段 (Segment/TCP) 或 数据报 (Datagram/UDP)	端口号 (16bit)	—	端到端可靠/不可靠传输
网络层	发到哪里? (Where)	包 (Packet)	IP地址 (32/128bit)	路由器	路由选路、分片重组
网络接口层	怎么发送? (How - physical)	帧 (Frame)	MAC地址 (48bit)	交换机/网桥	相邻节点帧传输、差错检测

1.3 数据封装与解封装





字段详解：

源端口 (Source Port, 16 bits)

- 发送方端口号 (0-65535)

目的端口 (Destination Port, 16 bits)

- 接收方端口号

序列号 (Sequence Number, 32 bits)

- TCP 对每个字节编号，序列号是该报文段中第一个数据字节的编号
- 初始序列号 (ISN) 是随机生成的 (防止旧连接串扰)
- ISN 随时间递增 (RFC 6528)，通过时钟 + 哈希生成

确认号 (Acknowledgment Number, 32 bits)

- 期望收到对方下一个报文段的第一个数据字节的序列号
- 即: "序号 < ACK号 的字节我都收到了"
- 仅当 ACK 标志有效时此字段有意义

数据偏移 (Data Offset, 4 bits)

- TCP头部长度，以 4 字节为单位
- 最小为 5 (20字节)，最大为 15 (60字节)

保留位 (Reserved, 4 bits)

- 保留供将来使用，目前设为 0

标志位 (Flags, 8 bits):

- | | |
|---------------------------------|-----------------------|
| CWR (Congestion Window Reduced) | - 拥塞窗口减小 (ECN) |
| ECE (ECN Echo) | - ECN 回显 |
| URG (Urgent) | - 紧急指针有效 (基本不用) |
| ACK (Acknowledgment) | - 确认号有效 (连接建立后始终为1) |
| PSH (Push) | - 提示接收方尽快交付应用层 (基本不用) |
| RST (Reset) | - 重置连接 (异常终止) |
| SYN (Synchronize) | - 同步序列号 (建立连接时使用) |
| FIN (Finish) | - 发送方数据发送完毕 (关闭连接) |

窗口大小 (Window Size, 16 bits)

- 接收方当前可用的接收缓冲区大小 (用于流量控制)
- 配合 window Scale Option (RFC 1323) 可达 1GB 窗口

- 最大原始值 65535 字节

校验和 (Checksum, 16 bits)

- 校验 TCP头部 + 数据 + 伪首部 (IP源/目的 + 协议号 + TCP长度)
- 必选字段 (IPv4中IP校验和只校验头部; UDP校验和可选)

紧急指针 (Urgent Pointer, 16 bits)

- 仅在 URG=1 时有意义, 指向紧急数据末尾

选项 (Options, 可变长度)

常见选项:

- MSS (Maximum Segment Size): 协商最大报文段大小 (通常在 SYN 中)
- Window Scale (窗口扩大因子): 窗口值 = $\text{window} * 2^{\text{shift}}$
- SACK Permitted / SACK: 选择性确认 (改进丢包恢复)
- Timestamps: 时间戳 (RTTM计算 + PAWS防回绕)
- NOP (No-Operation): 填充字节 (0x01), 用于对齐

面试题 2: TCP 序列号为什么要随机初始化 (ISN)?

答:

1. **防止旧连接的数据串扰:** 如果 ISN 固定 (如始终为0), 延迟在网络中的旧报文可能被误认为是新连接的合法数据。
2. **防止 TCP 序列号预测攻击:** 攻击者如果能预测序列号, 可以伪造 RST 或注入数据。
3. **实现:** ISN 通过一个基于时钟的算法生成, 每 4 微秒 ISN 加 1, 加上一个随机哈希, 使得 ISN 在约 4.55 小时内不会回绕。

2.2 三次握手

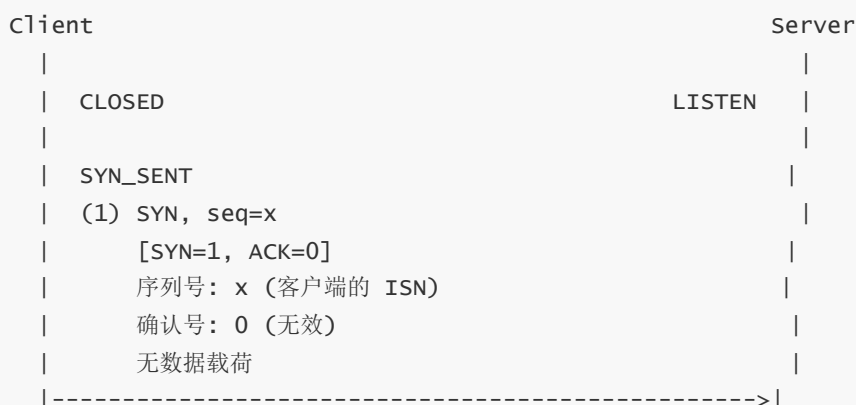
原理

TCP 三次握手 (Three-Way Handshake) 用于在客户端和服务端之间建立可靠的连接。握手过程完成了三个关键任务:

1. **同步初始序列号 (ISN)** — 双方互相告知各自的初始序列号
2. **协商参数** — MSS、窗口扩大因子、是否支持 SACK 等
3. **确认双方收发能力** — 确保双向通信通道畅通

流程图

三次握手 (Three-way Handshake)





状态变化:

Client: CLOSED -> SYN_SENT -> ESTABLISHED

Server: LISTEN -> SYN_RCVD -> ESTABLISHED

深入细节

Step 1: 客户端发送 SYN

- 客户端做的事情:
- 1. 生成初始序列号 x (基于时钟+随机)
 - 2. 构建TCP报文: SYN=1, seq=x, ACK=0
 - 3. 设置选项: MSS, window scale, SACK Permitted, Timestamp
 - 4. 发送
 - 5. 进入 SYN_SENT 状态
 - 6. 启动重传定时器
- 内核中: 将连接信息放入半连接队列 (syn queue)

Step 2: 服务器回复 SYN+ACK

- 服务器做的事情:
- 1. 收到 SYN, 验证合法性
 - 2. 生成自己的初始序列号 y
 - 3. 构建响应: SYN=1, ACK=1, seq=y, ack=x+1
 - 4. 设置选项: MSS, window scale, SACK Permitted (协商结果)
 - 5. 发送
 - 6. 进入 SYN_RCVD 状态
 - 7. 启动重传定时器, 放入半连接队列

ack=x+1 的含义:

"你发的 seq=x 的 SYN 我收到了, 期望你的下一个字节从 x+1 开始"

(SYN 报文占用一个序列号)

Step 3: 客户端回复 ACK

客户端做的事情：

1. 收到 SYN+ACK，验证 $ack == x+1$ （确认服务端收到了自己的SYN）
2. 构建响应：ACK=1, seq=x+1, ack=y+1
3. 该ACK可以携带数据（如HTTP请求）
4. 发送
5. 进入 ESTABLISHED 状态
6. 从半连接队列移到全连接队列（accept queue）

服务器收到ACK：

1. 验证 $ack == y+1$
2. 进入 ESTABLISHED 状态
3. 将连接从半连接队列移到全连接队列
4. accept() 系统调用可以取走此连接

面试题 3: 为什么是三次握手而不是两次或四次?

答: 这是最经典的面试题之一，必须从多角度回答。

角度看: 为什么不能是两次?

两次握手的问题场景 - 防止已失效的连接请求到达服务器：

1. 客户端第一次发送 SYN (seq=x)，但在网络中延迟了
2. 客户端超时，重发 SYN (seq=x')，这次成功建立了连接
3. 连接关闭后，那个延迟的旧 SYN (seq=x) 终于到达了服务器
4. 服务器以为是一个新连接请求，回复 SYN+ACK (seq=y, ack=x+1)
5. 如果是两次握手：
 - 服务器认为连接已建立（进入 ESTABLISHED）
 - 但客户端并没有发起连接！
 - 服务器傻等，白白占用资源
6. 如果是三次握手：
 - 客户端收到意想不到的 SYN+ACK，发送 RST 拒绝
 - 服务器收到 RST，知道这是个无效连接，释放资源

角度看: 为什么不能是四次?

三次握手已经能够：

- 客户端确认自己的发送能力 (SYN到服务器)
- 服务器确认自己的接收能力 (收到SYN) 和发送能力 (发送SYN+ACK)
- 客户端确认服务器的发送能力 (收到SYN+ACK) 和自己的接收能力

第四次是多余的: 服务器发出 SYN+ACK 后，客户端发出的 ACK 已经确认了双方通道畅通。

角度看: 信息论/信道建立

核心目标：双方交换 ISN

客户端知道服务端ISN：需要1.5个RTT（客户端->服务器->客户端）

服务端知道客户端ISN：需要1个RTT（客户端->服务器）

最少消息数：

- 客户端告诉服务端 "我的ISN是x"：1条消息（SYN）
- 服务端告诉客户端 "我的ISN是y，我知道你的ISN是x"：1条消息（SYN+ACK）
- 客户端告诉服务端 "我知道你的ISN是y"：1条消息（ACK）

最少3条！这就是三次握手的信息论基础。

面试题 4: SYN Flood 攻击是什么? 如何防范?

答:

- **原理:** 攻击者发送大量 SYN 报文 (伪造源IP)，服务器回复 SYN+ACK 后进入 SYN_RCVD 状态，但由于源IP是伪造的，服务器永远收不到最后的 ACK。半连接队列被打满，正常用户的 SYN 被丢弃。
- **防范:**
 1. **SYN Cookie** (Linux 默认开启): 收到 SYN 时不在半连接队列中分配资源，而是根据 (源IP, 源端口, 目的IP, 目的端口, 时间戳) 计算一个 Cookie 嵌入到 ISN 中。收到 ACK 时验证 Cookie 的合法性，通过则分配资源。
 2. **增大半连接队列:** `tcp_max_syn_backlog` 内核参数
 3. **缩短 SYN Timeout:** `tcp_synack_retries` 减少重试次数
 4. **SYN Proxy / 防火墙:** 由防火墙代答 SYN+ACK，等收到合法ACK再转发给真实服务器

项目应用

在实际项目中，三次握手相关的常见问题:

- **连接建立慢:** 检查网络 RTT，可能是跨地域部署导致。测量方法: `ping` 或 `tcprstat`。
- **SYN Cookie 误伤:** 高并发合法用户可能被 SYN Cookie 机制丢弃，调整 `tcp_max_syn_backlog` 或优化 `tcp_abort_on_overflow`。
- **端口耗尽:** 客户端大量短连接导致 TIME_WAIT 过多，端口不够 → 端口复用。

2.3 四次挥手

原理

TCP 是全双工通信，每个方向可以独立关闭。四次挥手 (Four-Way Wave) 用于优雅地释放连接。

流程图

四次挥手 (Four-Way Wave)

Client (主动关闭方)

|
|

ESTABLISHED

Server (被动关闭方)

|
|

ESTABLISHED



状态变化:

Client: ESTABLISHED -> FIN_WAIT_1 -> FIN_WAIT_2 -> TIME_WAIT -> CLOSED

Server: ESTABLISHED -> CLOSE_WAIT -> LAST_ACK -> CLOSED

深入细节

为什么是四次不是三次?

因为 TCP 是全双工的, 每个方向需要独立关闭:

- 客户端 FIN: 客户端->服务端方向关闭
- 服务器 ACK: 确认客户端方向关闭
- 服务器 FIN: 服务器->客户端方向关闭
- 客户端 ACK: 确认服务器方向关闭

中间的两步 (ACK + FIN) 可以合并吗? 在特定情况下可以! 如果服务器收到客户端的 FIN 后**没有数据要发送了**, 可以将 ACK 和 FIN 合并到同一个报文中发送 → 变成"三次挥手"。

但通常服务器收到 FIN 后还有数据在处理/发送中 (应用层还没调用 close), 所以先回 ACK, 等数据处理完毕再发 FIN, 这就是为什么通常看到四次。

TCP 半关闭状态:

半关闭 (Half-Close):

- 客户端调用 `shutdown(sockfd, SHUT_WR)` 只关闭写端
- 客户端仍可接收服务器发来的数据
- 服务器收到 `FIN` 后知道客户端不会再发数据, 但仍可发送

`close(sockfd)` vs `shutdown(sockfd, SHUT_WR)`:

- `close`: 减少fd引用计数, 计数为0时才发`FIN` (有其他进程共享fd时不发)
- `shutdown`: 立即发起关闭, 不依赖引用计数

面试题 5: 如果四次挥手过程中有报文丢失会怎样?

答:

1. **丢失第一个 FIN**: 客户端 `FIN_WAIT_1` 超时后重传 `FIN`。
2. **丢失第二个 ACK**: 客户端 `FIN_WAIT_1` 超时后重传 `FIN` (TCP 累计确认, `ACK` 本身不重传, 靠 `FIN` 的重传来触发对方重传 `ACK`)。
3. **丢失第三个 FIN**: 服务器 `LAST_ACK` 超时后重传 `FIN`。
4. **丢失第四个 ACK**: 服务器 `LAST_ACK` 超时后重传 `FIN`; 客户端 `TIME_WAIT` 中收到重传的 `FIN` 后, 重发 `ACK` 并重置 `2MSL` 定时器。

2.4 TIME_WAIT

原理

`TIME_WAIT` 是**主动关闭方**在发送最后一个 `ACK` 后进入的状态, 持续 `2MSL` (Maximum Segment Lifetime, 报文最大生存时间) 时长。

`TIME_WAIT` 持续: $2 * MSL$

`MSL` (RFC 793): 2 分钟 (Linux 实际: 60秒, 可通过 `/proc/sysctl` 调整)

Linux 实际 `TIME_WAIT`: 60秒 ($2 * 30s$ `MSL` 或 固定值)

```
$ cat /proc/sys/net/ipv4/tcp_fin_timeout
60
```

为什么需要 TIME_WAIT?

原因 1: 确保最后一个 ACK 能被对方收到

场景:

1. Client 发送 `FIN`, Server 回复 `ACK`
2. Server 发送 `FIN`, Client 回复 `ACK` (Client 进入 `TIME_WAIT`)
3. 如果 Client 的最后一个 `ACK` 丢失:
 - +- Server 在 `LAST_ACK` 超时后重传 `FIN`
 - +- Client 仍在 `TIME_WAIT` 中, 能收到并重传 `ACK`
 - +- 如果 Client 没有 `TIME_WAIT` 直接 `CLOSED`:
 - Server 重传 `FIN` -> Client 返回 `RST` -> Server 收到 `RST` 认为出错 (Server 调用 `close` 后 `read` 返回 0, 但如果收到 `RST` 则是错误)

原因 2: 让旧连接的所有报文在网络中消失

场景：

1. **Client** 和 **Server** 建立一个连接 (**src_port**=6666, **dst_port**=80)
2. 传输数据后, **Client** 主动关闭, 进入 **TIME_WAIT**
3. 如果 **Client** 立即以同样的端口对 (6666 -> 80) 建立新连接:
 - +-- 旧连接延迟在网络中的报文可能被新连接误收
 - +-- 特别是如果旧连接的序列号恰好落在新连接的接收窗口中
4. **TIME_WAIT** 等待 2MSL 确保了:
 - +-- 旧连接的所有报文 (包括重传的) 都已消失
 - +-- 网络中不会有属于旧连接的残留报文

TIME_WAIT 带来的问题

大量 **TIME_WAIT** 的危害：

1. 端口耗尽 (客户端场景):
 - 客户端作为主动关闭方发起大量短连接
 - 每个 **TIME_WAIT** 占一个本地端口约 60 秒
 - 可用端口范围: 32768 ~ 60999 (约 28000 个)
 - $28000 / 60s \approx 466$ 连接/秒 (瓶颈)
 - 超过则: **Cannot assign requested address (EADDRNOTAVAIL)**
2. 内存占用:
 - 每个 **TIME_WAIT** 连接占用一个 **tcp_timewait_sock** 结构体 (约 200字节)
 - 大量**TIME_WAIT**浪费内存 (虽然单个很小)
3. 连接数限制:
 - **TIME_WAIT** 连接也计入文件描述符限制? (不, **TIME_WAIT**已释放fd)
 - 但计入 **connttrack** 表 (如果使用 NAT)

如何处理 TIME_WAIT

```
# 方案1: 开启端口复用 (服务端)
# 允许在 TIME_WAIT 的端口上启动监听
# (用于重启服务器时避免 Address already in use)
int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

# 方案2: 开启端口复用 (客户端)
# 允许复用 TIME_WAIT 状态的端口建立新连接
# (用于客户端大量短连接场景)
$ echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse # 客户端复用TIME_WAIT端口

# 方案3: 快速回收 TIME_WAIT (Linux 4.12+ 已移除)
# 不要用! 在 NAT 环境下会导致问题
$ echo 1 > /proc/sys/net/ipv4/tcp_tw_recycle # 已废弃!

# 方案4: 减少 TIME_WAIT 时长
$ echo 30 > /proc/sys/net/ipv4/tcp_fin_timeout # 默认60秒

# 方案5: 增加可用端口范围
$ echo "1024 65535" > /proc/sys/net/ipv4/ip_local_port_range
```

方案6：使用连接池/长连接（根本解决方案）
复用连接而不是频繁创建新连接

面试题 6: SO_REUSEADDR 和 tcp_tw_reuse 的区别？

答:

特性	SO_REUSEADDR	tcp_tw_reuse
层面	Socket 选项 (应用层)	内核参数 (系统级)
用途	允许 bind() 到 TIME_WAIT 的端口	允许作为客户端复用 TIME_WAIT 端口发起新连接
典型场景	服务端重启	客户端大量短连接
安全条件	仅bind检查	需要 TCP Timestamps 开启，且新连接的 timestamp > 旧连接
风险	可能收到旧连接的数据 (极低概率)	NAT 环境可能有问题

2.5 CLOSE_WAIT

原理

CLOSE_WAIT 是被动关闭方在收到 FIN、回复 ACK 后进入的状态。这个状态表示: 对方已经关闭了连接，我方应该调用 close() 来彻底关闭。

CLOSE_WAIT 产生的原因

正常流程：
Server 收到 Client 的 FIN -> ACK -> 进入 CLOSE_WAIT
Server 应用层调用 close()/shutdown() -> 发送 FIN -> 进入 LAST_ACK
Server 收到 Client 的 ACK -> CLOSED

异常（CLOSE_WAIT 堆积）：
Server 收到 Client 的 FIN -> ACK -> 进入 CLOSE_WAIT
Server 应用层 ... 一直没有调用 close()！
-> CLOSE_WAIT 一直存在，直到进程退出

- 原因：
- 1. 代码 bug - 忘记调用 close()
 - 2. 代码逻辑不完善 - 错误处理路径遗漏了 close()
 - 3. 线程阻塞 - 处理请求的线程卡住，无法执行到 close()
 - 4. 未处理半关闭 - 对方 shutdown(SHUT_WR) 后我方仍需 close()

如何排查 CLOSE_WAIT

```
# 统计各状态的 TCP 连接数
$ netstat -ant | awk '{print $6}' | sort | uniq -c | sort -rn
    50 ESTABLISHED
    30 TIME_WAIT
    12 CLOSE_WAIT      # <-- 不正常！通常应该很少
     5 LISTEN

# 查看 CLOSE_WAIT 连接的详情
$ netstat -antp | grep CLOSE_WAIT
tcp  0  0  192.168.1.1:8080  10.0.0.5:54321  CLOSE_WAIT  12345/java

# 找到进程 PID=12345
$ strace -p 12345 -e trace=close,shutdown
# 观察是否有 close/shutdown 调用

$ lsof -p 12345 | grep TCP
# 查看进程打开的所有TCP连接

# 如果是Java
$ jstack 12345
# 分析线程栈，找到未close的连接对应的处理线程
```

面试题 7: CLOSE_WAIT 很多怎么办？

答:

1. **定位进程:** `netstat -antp | grep CLOSE_WAIT` 找到持有这些连接的进程。
2. **检查代码:** 找到服务端处理网络IO的代码，确认：
 - 当 `read()` 返回 0 (对端关闭) 时，是否正确调用了 `close()`
 - 异常处理路径 (如超时、业务错误) 是否也调用了 `close()`
 - 是否使用 RAII (如 C++ 的 `unique_fd`) 或 try-with-resources (Java) 自动关闭
3. **检查线程状态:** 是否有线程卡在某个操作上无法继续执行到 `close`
4. **临时恢复:** 重启进程 (但应找到根因)

2.6 滑动窗口

原理

TCP 的**滑动窗口 (Sliding Window)** 是流量控制的核心机制。窗口大小由接收方在 TCP 头部中的 `window size` 字段告知发送方，告诉发送方: "我还能接收这么多字节"。

滑动窗口示意

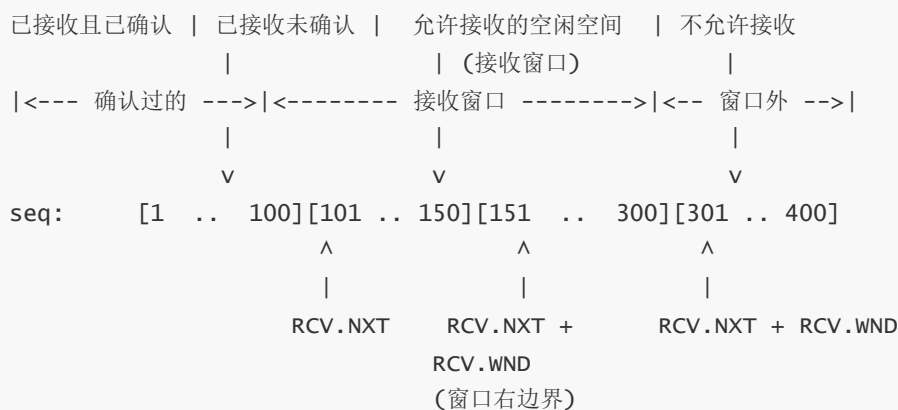
发送方维护的发送窗口：

已发送且已确认		已发送未确认		允许发送但未发送		不允许发送
		(发送窗口)				
<--- 确认过的	----	<----- 发送窗口	----->	<-- 窗口外	-->	



SND.UNA - 发送窗口左边界 (第一个未确认的字节)
 SND.NXT - 下一个要发送的字节
 SND.WND - 发送窗口大小 (由接收方通告)

接收方维护的接收窗口:



RCV.NXT - 接收窗口左边界 (期望的下一个字节)
 RCV.WND - 接收窗口大小 (空闲缓冲区大小)

窗口滑动:

- 当 SND.UNA 的确认到达时, 窗口右移 (发送更多数据)
- 当 RCV.NXT 之前的数据被应用程序读取后, 窗口右移 (通告更大的窗口给对端)

面试题 8: 什么是零窗口? 如何解决?

答:

- **零窗口 (Zero Window):** 接收方通告窗口大小为 0, 告诉发送方 "我满了, 别发了"。这通常发生在接收方应用层处理太慢, 没有及时从接收缓冲区读取数据。
- **解决方案:**
 1. 优化应用层处理速度
 2. 增大接收缓冲区: `setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &bufsize, ...)`
 3. 系统级调优: `net.core.rmem_max`, `net.ipv4.tcp_rmem`
- **零窗口探测:** 发送方收到零窗口后, 会周期性地发送**零窗口探测报文 (Zero Window Probe)** (1字节数据), 查看接收方窗口是否重新打开。

2.7 流量控制

原理

流量控制 (Flow Control) 是一种端到端的机制: **接收方通过调整窗口大小来控制发送方的发送速率**, 防止发送方发送太快导致接收方来不及处理而丢包。

流量控制与拥塞控制的区别

流量控制 (Flow Control)	拥塞控制 (Congestion Control)
端到端的 (发送方 <-> 接收方)	全局性的 (发送方 <-> 网络)
防止接收方缓冲区溢出	防止网络拥塞 (路由器缓冲区溢出)
由接收方窗口 (rwnd) 控制	由拥塞窗口 (cwnd) 控制
关注点是对方的处理能力	关注点是网络的承载能力

实际发送窗口 = $\min(\text{rwnd}, \text{cwnd})$

其中: rwnd = 接收方通告的窗口 (流量控制)

cwnd = 拥塞窗口 (拥塞控制)

工作机制

接收方:
空闲缓冲区大小 = SO_RCVBUF - (接收但未被应用读取的数据量)
这个空闲缓冲区大小就是通告给发送方的窗口 (rwnd)

发送方:
已发送未确认的数据量 = $\text{SND.NXT} - \text{SND.UNA}$ (即 "飞在空中的数据")
可以发送的数据量 = $\text{rwnd} - (\text{SND.NXT} - \text{SND.UNA})$
如果这个值 ≤ 0 : 停止发送

窗口更新:
接收方应用层 $\text{read}()$ 取了数据 -> 空闲缓冲区变大
-> 发送新的 ACK 携带更大的窗口值 -> 发送方可以发送更多数据

面试题 9: 如果接收方通告窗口一直很大, 为什么还需要拥塞控制?

答: 因为接收方处理能力只是限制因素之一, 网络的拥塞状况是另一个限制因素。即使接收方能处理 10MB/s, 如果中间路由器的带宽只有 1MB/s, 发送方按 10MB/s 发送会导致大量丢包。拥塞控制的作用就是探测网络的可用带宽, 防止网络拥塞。

2.8 拥塞控制

原理

TCP 拥塞控制 (Congestion Control) 是 TCP 最核心的算法之一, 由发送方根据网络拥塞状况动态调整发送速率。核心变量是**拥塞窗口 (cwnd)**。

四大算法

Tahoe/Reno 拥塞控制算法的演化（经典算法）

TCP Tahoe (Jacobson 1988):

- Slow Start
- Congestion Avoidance
- Fast Retransmit (触发后进入 Slow Start)

TCP Reno (Jacobson 1990, 至今最广泛使用的基础算法):

- Slow Start
- Congestion Avoidance
- Fast Retransmit (触发后进入 Fast Recovery)
- Fast Recovery

TCP NewReno (改进Reno在多个丢包时的表现)

TCP BBR (Google, 基于带宽和RTT的模型, 而非丢包)

TCP CUBIC (Linux 默认, 基于三次函数的窗口增长)

慢启动 (Slow Start)

目的: 快速探测网络的可用带宽

机制:

 cwnd 初始值: 通常是 $10 * MSS$ (Linux 3.0+)
 (旧版本: $1 MSS$, 通过 `initcwnd` 调整)

 每收到一个 ACK: $cwnd += 1 (MSS)$
 (等效于每经过一个 RTT, cwnd 翻倍)

 指数增长: 1, 2, 4, 8, 16, 32 ...
 直到 $cwnd \geq ssthresh$ (慢启动阈值) 或发生丢包

慢启动阈值 (ssthresh):

- 初始值: 通常设为无穷大 (初始时允许足够指数增长)
- 发生超时丢包后: $ssthresh = \max(cwnd/2, 2*MSS)$
- $cwnd \geq ssthresh$ 时, 进入拥塞避免阶段

拥塞避免 (Congestion Avoidance)

目的: 缓慢增加 cwnd, 小心翼翼地接近网络带宽极限

机制 (AIMD: Additive Increase):

 每经过一个 RTT: $cwnd += 1 (MSS)$

 线性增长: 每个 ACK 增加: $cwnd += MSS * (MSS / cwnd)$

 也就是说, 每过一个 RTT, cwnd 只增加 1 个 MSS

快速重传 (Fast Retransmit)

目的：在超时之前快速发现并重传丢包

机制：

当发送方连续收到 3 个重复的 ACK (DupACK) 时：
(dupACKcount >= 3)

判定：对应的包丢失了！
立即重传该包（不等待超时）

为什么是 3 个重复 ACK？

1 个可能是乱序，2 个也可能是乱序，3 个几乎可以确定是丢包

TCP Reno：收到 3 个 DupACK -> Fast Retransmit + Fast Recovery
TCP Tahoe：收到 3 个 DupACK -> Fast Retransmit + Slow Start（更保守）

快速恢复 (Fast Recovery)

目的：快速重传后，不让 cwnd 降得太多，快速恢复到合理水平

机制（TCP Reno）：

收到 3 个重复 ACK 时：

1. ssthresh = max(cwnd / 2, 2 * MSS)
2. cwnd = ssthresh + 3 * MSS（不是降到1!）
3. 重传丢失的包
4. 每收到一个重复 ACK：cwnd += 1 (MSS)
（因为每个 DupACK 表示有一个包被接收方收到了）
5. 收到新数据的 ACK 时：
cwnd = ssthresh（恢复到拥塞避免水平）
进入 Congestion Avoidance

（新数据ACK：确认的序列号大于之前确认的序列号，
意味着丢失的包已被重传并收到）

如果快速恢复期间也超时了：
-> 进入 Slow Start
ssthresh = max(cwnd/2, 2*MSS), cwnd = 1 MSS

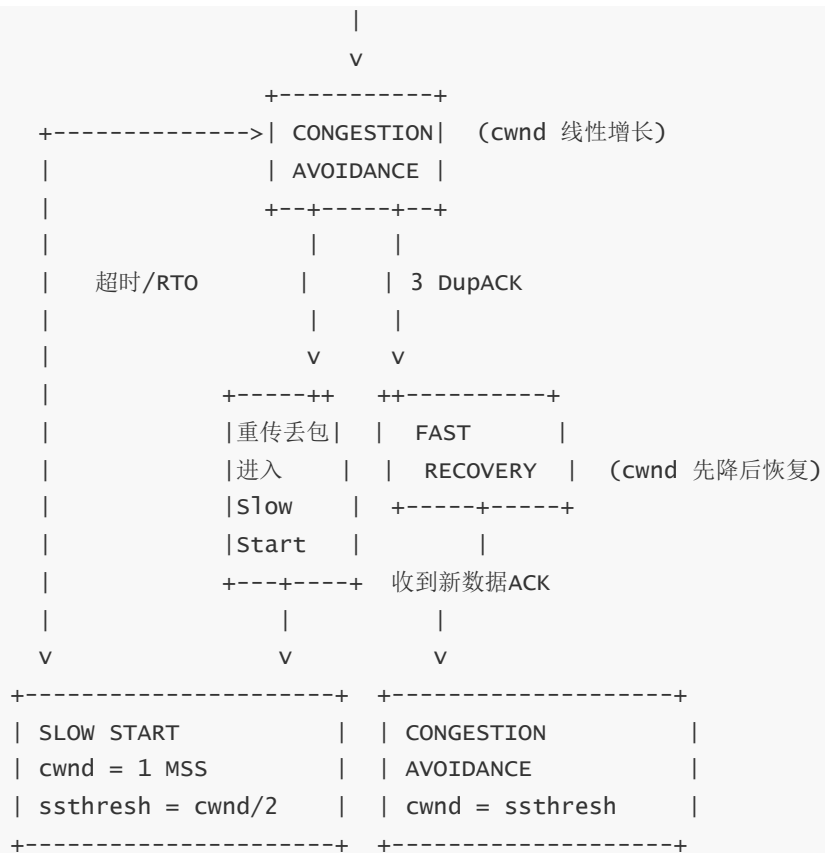
拥塞控制状态机

TCP Reno 拥塞控制状态机

+-----+
| SLOW |
| START | (cwnd 指数增长)
+-----+

|

cwnd >= ssthresh
或 收到 DupACK (Tahoe)



触发 slow start 的条件：

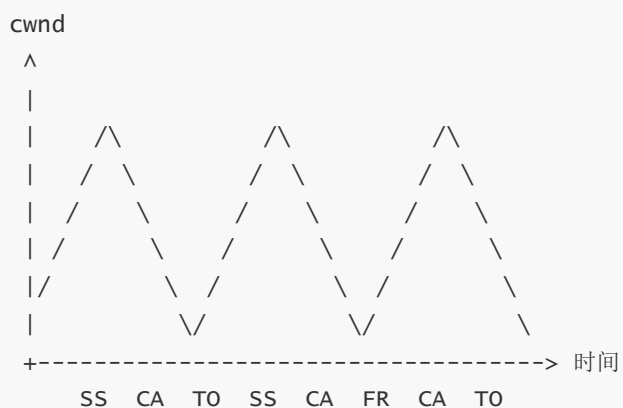
- 连接刚开始时
- 超时重传 (RTO) 发生时

触发 Fast Retransmit + Fast Recovery:

- 连续收到 3 个 DupACK (且不是超时)

面试题 10: 请画出 TCP Reno 拥塞窗口随时间变化的曲线图 (锯齿图)

答:



SS = slow start (指数增长)

CA = Congestion Avoidance (线性增长)

TO = Timeout (超时, 回到Slow Start)

FR = Fast Recovery (3 DupACKs, 窗口折半)

关键特征:

- 慢启动：指数增长（曲线陡）
- 拥塞避免：线性增长（曲线平缓）
- 超时丢包：窗口降到 1，回到慢启动
- 3 DupACKs：窗口减半，进入快速恢复后回到拥塞避免（曲线上的小缺口 - 减半后继续线性增长）

这就是经典的 "TCP 锯齿图"

面试题 11: TCP CUBIC (Linux 默认) 和 TCP Reno 有什么区别?

答:

- **Reno**: 窗口增长是线性的 (每RTT +1), 在高带宽长延迟 (BDP大) 网络中恢复慢, 带宽利用率低。
- **CUBIC**: 窗口增长是**三次函数**。在丢包后, CUBIC 将窗口作为距离上次丢包时间的函数:
 - 刚丢包后: 窗口稳定在 W_{max} 附近, 增长很慢 (探测)
 - 远离丢包点: 窗口激进增长 (利用可用带宽)
 - 接近 W_{max} : 窗口增长又变慢 (小心探测)
 - 超越 W_{max} : 更高带宽可用时, 更快增长

CUBIC 不依赖 RTT 公平性 (Reno 的 RTT 公平性问题: RTT 小的连接增长比大RTT连接快), 在 BDP 大的网络中表现更好。

面试题 12: BBR 是什么? 和传统拥塞控制的区别?

答:

- **BBR (Bottleneck Bandwidth and Round-trip propagation time)**: Google 开发的拥塞控制算法。
- **核心思想**: 不再以**丢包**作为拥塞信号, 而是通过持续测量网络的瓶颈带宽 (Bottleneck Bandwidth) 和最小 RTT (Round-Trip propagation time) 来确认网络的承载能力。
- **关键发现**: 传统算法 (如 CUBIC) 通过制造丢包来"发现"带宽, 在有 bufferbloat (大缓冲) 的网络中会在缓冲区填满后才丢包, 导致超高延迟。BBR 在丢包之前就探测到实际带宽上限。
- **优势**: 高吞吐量 + 低延迟 (不需要填满缓冲区)。
- **问题**: 在某些场景可能抢占太多带宽, 不公平。

项目应用

拥塞控制在实际项目中的应用:

1. **选择合适的拥塞控制算法**: 大文件传输用 BBR, 通用场景用 CUBIC。 `sysctl net.ipv4.tcp_congestion_control`。
2. **CDN/视频流**: BBR 减少缓冲和卡顿。
3. **跨数据中心**: 高 BDP 网络用 BBR 获取更好吞吐。
4. **监控丢包和重传**: `netstat -s | grep retrans` 查看重传统计。

2.9 Nagle 算法

原理

Nagle 算法 (RFC 896) 是为了解决**小包问题 (Silly Window Syndrome)** 而设计的。在一个 TCP 连接中, 如果应用程序频繁发送很小的数据块 (如每次发送 1 字节), TCP 头部 40 字节 + 1 字节数据 = 4100% 的头部开销, 网络效率极低。

Nagle 算法规则:

```
if (有未确认的数据 && 新数据量 < MSS) {  
    等待;    // 不立即发送  
    // 条件满足时发送:  
    // 1. 积累到 MSS 大小  
    // 2. 或收到了之前数据的 ACK  
} else {  
    立即发送;  
}
```

简而言之: 任何时候最多只能有一个未确认的小报文段

什么时候应该禁用 Nagle?

```
// 禁用 Nagle (设置 TCP_NODELAY)  
int opt = 1;  
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &opt, sizeof(opt));
```

需要禁用的场景:

- **实时性要求高的场景:** 游戏、股票行情、语音通话
- **交互性应用:** SSH 终端 (每个按键都要立刻发送)
- **HTTP 请求/响应:** 请求通常 < MSS, Nagle 会导致延迟 (等ACK)
- **小数据包+需要立刻响应:** 如 Redis 命令

面试题 13: Nagle 算法和 Delayed ACK 同时启用会有什么问题?

答: 这会导致经典的"双延迟"问题:

1. Nagle 算法: 有未确认的小包时不发新数据, 等待 ACK
 2. Delayed ACK: 收到数据后不立刻回复 ACK, 等待 40-200ms (期望有数据要发送, 可以捎带 ACK)
 3. 死锁: Nagle 等 Delayed ACK, Delayed ACK 等 Nagle 发送数据 (捎带)
 4. 解决方案: 对于请求-响应模式的应用, 设置 `TCP_NODELAY` 禁用 Nagle
-

2.10 粘包拆包

原理

TCP 是**面向字节流 (byte stream)** 的协议，它不维护消息边界。应用程序调用 `write` 两次发送两个消息，TCP 可能：

- 把它们合并成一段发送 (粘包)
- 把一个大的消息拆成多段发送 (拆包)

粘包/拆包的场景：

```

应用程序:      write("HELLO")    write("WORLD")
                |                  |
TCP发送缓冲区: [H][E][L][L][O][W][O][R][L][D]
                |                  |
网络传输:      |---- Segment 1 ----|---- Segment 2 ----|
                [H][E][L][L]      [O][W][O][R][L][D]

```

接收方 read 可能读到的:

情况1 (正常): 读第一次 -> "HELLO", 读第二次 -> "WORLD"

情况2 (粘包): 读一次 -> "HELLOWORLD" (两个消息合并)

情况3 (拆包): 读第一次 -> "HEL", 读第二次 -> "LOWORLD"

情况4 (混合): 读第一次 -> "HELLOW", 读第二次 -> "ORLD"

原因：

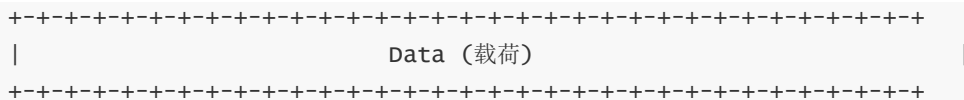
- Nagle 算法 (合并小包)
- MSS 限制 (大包被拆分)
- 接收方读取时机不确定
- TCP 缓冲区大小限制

解决方案

方案	原理	优点	缺点	适用场景
固定长度	每个消息固定 N 字节	实现简单	浪费带宽，不灵活	协议中每个消息类型大小固定
分隔符	以特殊字符分隔消息 (如 <code>\r\n</code>)	可读性好，灵活	需转义，内容不能含分隔符	HTTP, SMTP, Redis
长度前缀 (Length-Prefixed)	消息头包含消息体长度	高效，通用	需要定义头格式，避免不完整读	大多数二进制协议
自描述协议	使用成熟的序列化方案	最完善	依赖第三方库	Protobuf, Thrift, FlatBuffers

```
// 长度前缀方案示例（最常见）
// 协议格式：[4字节长度（网络字节序）][N字节消息体]

// 发送
int send_message(int fd, const char *data, uint32_t len) {
```

字段详解：

源端口： 可选（不需要回复时可设为 0）

目的端口： 接收方端口号

长度: UDP头部+数据的字节数 (最小 8)

校验和: 可选 (IPv4), 必选 (IPv6)

特点:

- 无连接：不需要握手，直接发送
- 不可靠：不保证到达、顺序、不重复
- 无拥塞控制：可以任意速率发送（不受 `cwnd` 限制）
- 面向数据报：保留消息边界（一次 `sendto` = 一次 `recvfrom`）

3.2 TCP vs UDP 对比表

维度	TCP	UDP
连接	面向连接 (需要握手)	无连接 (直接发送)
可靠性	可靠 (确认+重传)	不可靠 (尽力而为)
顺序	保证 (序列号+重排)	不保证
边界	字节流 (无消息边界)	数据报 (保留消息边界)
头部开销	20-60 字节	8 字节
传输速度	慢 (受拥塞控制限制)	快 (无拥塞控制)
流量控制	有 (滑动窗口)	无
拥塞控制	有 (cwnd)	无 (应用层自行控制)
适用场景	HTTP, FTP, SSH, 数据库	DNS, VoIP, 视频直播, 游戏
广播/组播	不支持	支持
内核队列	连接队列 (accept queue)	接收缓冲区 (按数据报)

3.3 何时使用 UDP

UDP 适用的场景:

1. **实时性 > 可靠性**: 音视频通话 (VoIP), 直播 (错过几个包无所谓, 但不能等重传)
2. **简单查询/响应**: DNS (一个包能装下的请求/响应, 超时了重发就行)
3. **广播/组播**: TCP 不支持, 如局域网服务发现
4. **极高吞吐 + 可容忍丢包**: 部分游戏状态同步
5. **自定义传输协议**: QUIC (HTTP/3), KCP, Enet — 在 UDP 上实现自定义的可靠/拥塞控制

3.4 在 UDP 上实现可靠性

QUIC 协议 (HTTP/3 的基础) 在 UDP 上实现了:

- 可靠传输 (类似 TCP 的 ACK)
- 0-RTT 握手 (某些场景)
- 多路复用 (无队头阻塞)
- 前向纠错 (FEC)
- 连接迁移 (切换网络不断连)

但注意: 在 UDP 上实现可靠性 > 直接用 TCP 的原因是:

QUIC 可以定制拥塞控制、避免 TCP 的队头阻塞、减少握手延迟等

而不是因为 "UDP 更快" 这个简单认知

面试题 15: 为什么游戏中常用 UDP?

- 答:
- 1. **实时性:** 游戏每帧 (~16ms) 需要更新状态, TCP 超时重传可能导致数据延迟数百毫秒, 对手位置信息已过时。
 - 2. **容忍丢包:** 最新的位置/状态更重要, 旧的丢失数据没有必要重传。
 - 3. **自定义控制:** 游戏可以在 UDP 上根据游戏类型定制传输策略 (重要事件可靠发送, 高频位置更新不可靠发送)。
 - 4. **队头阻塞:** TCP 的队头阻塞 (丢包导致后续包全部阻塞) 对游戏是致命的。

4. IP

4.1 IP 头部格式

IPv4 Header 格式 (20-60 字节)

0	1	2	3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Version IHL DSCP ECN Total Length (16 bits)			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Identification (16 bits) Flags Fragment Offset			
(13 bits)			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Time to Live (TTL, 8 bits) Protocol (8 bits) Header			
(6=TCP, 17=UDP) Checksum			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Source IP Address (32 bits)			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Destination IP Address (32 bits)			
+-----+-----+-----+-----+-----+-----+-----+-----+			
Options (0-40 bytes, 可变)			
+-----+-----+-----+-----+-----+-----+-----+-----+			

关键字段:

Version (4 bits): 4 = IPv4, 6 = IPv6

IHL (4 bits): IP 头部长度 (4字节为单位), 最小 5 (20字节)

Total Length (16 bits): IP 数据报总长度 (头部 + 数据), 最大 65535

TTL (8 bits): 每经过一个路由器减1, 减到0则丢弃 (防循环)
Protocol (8 bits): 上层协议: 1=ICMP, 6=TCP, 17=UDP
Identification + Flags + Fragment Offset: IP 分片相关

4.2 IP 分片

MTU (Maximum Transmission Unit): 链路层最大传输单元, 以太网 = 1500 字节

当 IP 数据报 > MTU 时需要分片:

原始数据报: [IP Header | Data: 4000 bytes]

MTU = 1500, 需要分成 3 片

分片 1: [IP Header (MF=1, Offset=0)] [数据 0-1479]
分片 2: [IP Header (MF=1, Offset=185)] [数据 1480-2959]
分片 3: [IP Header (MF=0, Offset=370)] [数据 2960-3999]

注意:

- 分片在目标主机上重组 (不是路由器)
- Fragment Offset 以 8 字节为单位 (所以 offset=185 表示 $185 \times 8 = 1480$ 字节)
- MF (More Fragments) = 1 表示还有更多分片
- 如果任一分片丢失, 整个数据报被丢弃

IPv4 vs IPv6 分片:

- IPv4: 路由器可以分片
- IPv6: 路由器不分片 (需要 PMTUD - Path MTU Discovery)

面试题 16: TCP 需要处理 IP 分片吗?

答: 理想情况下 TCP 不需要。TCP 在建立连接时通过 MSS (Maximum Segment Size) 协商确保每个 TCP 报文不超过 MTU 限制, 从而避免 IP 层分片。MSS = MTU - IP_Header - TCP_Header = 1500 - 20 - 20 = 1460 (典型值)。

但如果路径中的 MTU 比预期的更小 (如隧道封装), 可能触发 IP 分片。这时 TCP 可以通过 **PMTUD (Path MTU Discovery)** 机制动态调整 MSS。设置 IP 头中的 DF (Don't Fragment) 标志, 如果中间路由器因为 MTU 不够需要分片, 会返回 ICMP "需要分片" 消息, 发送方据此调整 MSS。

5. ARP

5.1 ARP 工作原理

ARP (Address Resolution Protocol) 用于在局域网中将 IP 地址解析为 MAC 地址。这是因为数据链路层 (以太网) 需要 MAC 地址来传输帧。

ARP 工作流程

主机A (192.168.1.5) 想和 主机B (192.168.1.10) 通信
(A知道B的IP, 但不知道B的MAC)

A	B

```

| (1) ARP Request (广播) |
|   who has 192.168.1.10? |
|   Tell 192.168.1.5 |
|   [dst MAC: FF:FF:FF:FF:FF:FF] |
|----->|
|           (所有主机都收到，但只有B回复)|
|
|           (2) ARP Reply (单播) |
|           192.168.1.10 is at |
|           00:1A:2B:3C:4D:5E |
|           [dst MAC: A的MAC] |
|<-----|
|
| A 将 (192.168.1.10 -> 00:1A:2B:3C:4D:5E) |
| 存入 ARP 缓存 |
|
| (3) 用获取的 MAC 地址发送数据 |
|----->|

```

5.2 ARP 缓存

```

# 查看 ARP 缓存
$ arp -a
? (192.168.1.1) at aa:bb:cc:dd:ee:ff [ether] on eth0
? (192.168.1.10) at 00:1a:2b:3c:4d:5e [ether] on eth0

# 查看更多信息
$ ip neigh show
192.168.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
192.168.1.10 dev eth0 lladdr 00:1a:2b:3c:4d:5e STALE

```

ARP 缓存状态 (Linux):

- REACHABLE: 确认可到达 (最近通信过)
- STALE: ARP 条目老化, 待下次使用时验证
- DELAY: 等待确认
- PROBE: 正在发送 ARP 请求确认
- FAILED: 解析失败

5.3 ARP 欺骗

ARP Spoofing (ARP 欺骗):

攻击者 C 伪造 ARP Reply:
 "192.168.1.1 (网关) is at C的MAC"

A 收到后将错误的映射存入 ARP 缓存
 A 之后发给网关的数据都送到了 C
 C 成为中间人 (Man-in-the-Middle)

防范:

- 静态 ARP 条目 (小网络)

- ARP 检测工具 (arpwatch)
- 交换机端口安全 (Port Security)
- DHCP Snooping + DAI (Dynamic ARP Inspection)

面试题 17: 跨网段通信时需要 ARP 吗?

答: 不需要 ARP 解析目标主机的 MAC。跨网段通信时:

1. 源主机发现目标 IP 不在同一网段
2. 源主机 ARP 解析**网关 (默认路由器)** 的 MAC 地址
3. 数据帧的目标 MAC 是网关的 MAC，目标 IP 是最终目标的 IP
4. 网关收到后，根据路由表转发到下一跳，逐跳 ARP 解析

6. DNS

6.1 DNS 解析流程

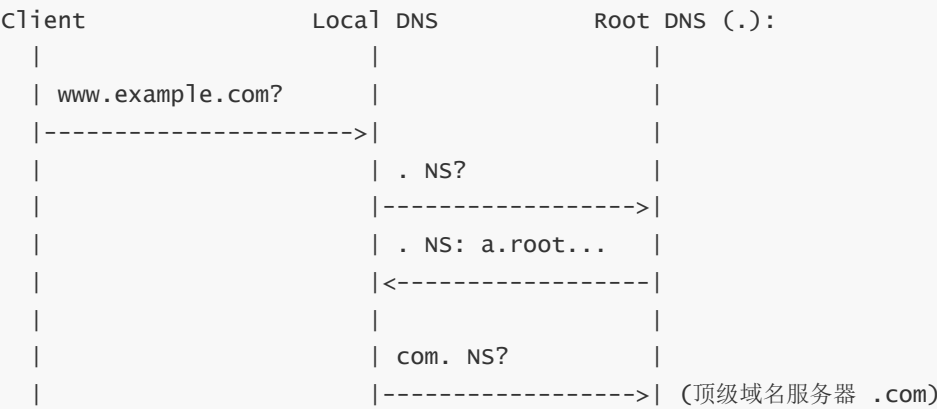
DNS 域名解析完整流程

用户在浏览器输入: `www.example.com`

- ```
+--- 1. 检查浏览器DNS缓存 (chrome://net-internals/#dns)
|
v miss
+--- 2. 检查操作系统DNS缓存 (ipconfig /displaydns 或 systemd-resolved)
|
v miss
+--- 3. 检查本地 hosts 文件 (/etc/hosts 或 C:\Windows\System32\drivers\etc\hosts)
|
v miss
+--- 4. 向本地DNS服务器 (LDNS，通常是运营商提供的) 发起递归查询
```

本地DNS服务器 (如 8.8.8.8):

- ```
|
+--- 5. 检查自身缓存
|
v  miss or expired
+--- 6. 向根域名服务器发起迭代查询
```



```

| | com. NS: a.gtld... |
| | <----- |
| | |
| | example.com NS? |
| | -----> | (权威域名服务器)
| | example.com NS: |
| | ns1.example.com |
| | <----- |
| | |
| | www.example.com? |
| | -----> | (ns1.example.com)
| | A: 93.184.216.34 |
| | <----- |
| |
| A: 93.184.216.34 |
| <----- |

```

递归查询: Client -> LDNS (中间步骤由LDNS完成)
 迭代查询: LDNS -> Root -> TLD -> Auth (每步都是LDNS自己问)

6.2 DNS 记录类型

类型	含义	示例
A	IPv4 地址	example.com. 300 IN A 93.184.216.34
AAAA	IPv6 地址	example.com. 300 IN AAAA 2606:2800:220:1:248:1893:25c8:1946
CNAME	别名/规范名	www.example.com. 300 IN CNAME example.com.
MX	邮件服务器	example.com. 300 IN MX 10 mail.example.com.
NS	域名服务器 (权威)	example.com. 86400 IN NS ns1.example.com.
PTR	反向解析 (IP->域名)	34.216.184.93.in-addr.arpa. IN PTR example.com.
TXT	文本记录 (验证/SPF 等)	example.com. 300 IN TXT "v=spf1 include:_spf.google.com ~all"
SRV	服务定位	_sip._tcp.example.com. 3600 IN SRV 10 60 5060 sip.example.com.
SOA	权威记录起始	包含主DNS、管理员邮箱、序列号、刷新时间等

6.3 DNS 缓存

- 各级缓存:
1. 浏览器缓存: Chrome 默认缓存 60 秒
 2. 操作系统缓存:
 - windows: DNS Client 服务缓存
 - Linux: nscd 或 systemd-resolved 缓存

- 3. 本地 DNS 服务器缓存：递归 DNS 服务器（如 8.8.8.8）缓存
- 4. TTL 控制：每条 DNS 记录都有 TTL (Time To Live)
 - 短 TTL：快速切换（如故障转移），但解析慢
 - 长 TTL：解析快，但变更慢

```
DNS 缓存查看和清理：
$ ipconfig /displaydns (Windows)
$ ipconfig /flushdns (Windows)
$ systemd-resolve --flush-caches (Linux, systemd-resolved)
$ sudo systemctl restart nscd (Linux, nscd)
```

面试题 18: 一个域名解析过程涉及哪些层面的缓存? 如果域名IP变了, 多久全局生效?

- 答:
- 1. 涉及缓存层面: 浏览器缓存 -> OS缓存 -> hosts文件 -> LDNS缓存 -> 各级权威DNS缓存
 - 2. 生效时间: 取决于 TTL。如果 TTL=300 秒，理论上最多 5 分钟。但各层缓存的 TTL 处理不完全一致 (某些运营商可能修改 TTL)，实际可能需要更长时间。Google DNS (8.8.8.8) 严格遵循 TTL。

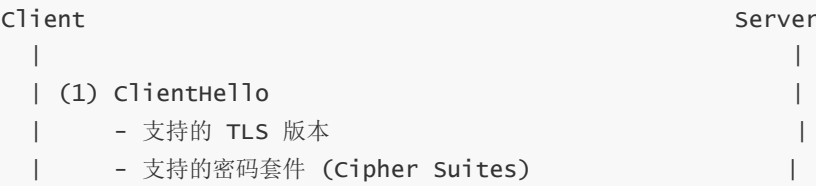
7. HTTP/HTTPS

7.1 HTTP 版本对比

特性	HTTP/1.0	HTTP/1.1	HTTP/2 (H2)	HTTP/3 (H3)
发布时间	1996 (RFC 1945)	1999 (RFC 2616)	2015 (RFC 7540)	2022 (RFC 9114)
连接	短连接 (默认)	长连接 (Keep-Alive 默认)	单连接多路复用	单连接多路复用
队头阻塞	严重	有 (HTTP层面)	有 (TCP层面)	无 (基于QUIC)
头部压缩	无	无	HPACK	QPACK
二进制格式	否 (文本)	否 (文本)	是	是
服务端推送	无	无	支持	支持
传输层协议	TCP	TCP	TCP	UDP (QUIC)
TLS	单独	单独	通常强制	内置 (QUIC TLS 1.3)
连接迁移	无	无	无	支持 (Connection ID)

7.2 HTTPS 握手 (TLS 1.2)

完整 TLS 1.2 握手流程



```

|   - 客户端随机数 (Client Random) |
|   - 支持的压缩方法 |
|   - 扩展 (SNI, ALPN, ...) |
|----->|
|
|           (2) ServerHello |
|           - 选择的 TLS 版本 |
|           - 选择的密码套件 |
|           - 服务器随机数 (Server Random) |
|           (3) Certificate |
|           - 服务器证书链 |
|           (4) ServerHelloDone |
|<-----|
|
| [Client 验证证书] |
| - 证书链验证 (CA 签名) |
| - 证书有效期, 域名匹配 (SAN/CN) |
| - 证书吊销检查 (CRL / OCSP) |
|
| (5) ClientKeyExchange |
|   - RSA: 用服务器公钥加密 Pre-Master Secret |
|   - DH(E): 客户端 DH 公钥参数 |
| (6) ChangeCipherSpec |
|   - "后续消息将使用协商的密钥加密" |
| (7) Finished (加密) |
|   - 验证握手的完整性 |
|----->|
|
|           [Server 计算 Master Secret] |
|           (8) ChangeCipherSpec |
|           (9) Finished (加密) |
|<-----|
|
| === 安全通道建立, 开始传输加密的应用数据 === |

```

密钥计算:

```

Pre-Master Secret (客户端生成, 服务端解密/协商得出)
|
v
Master Secret = PRF(Pre-Master, "master secret",
                    ClientRandom + ServerRandom)
|
v
Session Keys = PRF(Master Secret, "key expansion",
                    ServerRandom + ClientRandom)
-> Client Write Key, Server Write Key, IVs, MAC keys

```

7.3 TLS 1.3 握手 (简化)



7.4 常见 HTTP 头部

分类	头部	说明
请求	Host	目标主机 (HTTP/1.1 必选)
	User-Agent	客户端信息
	Accept / Accept-Encoding	可接受的响应类型/编码
	Authorization	认证信息
	Cookie	发送 Cookie
	Referer	来源页面 URL
	Content-Type	请求体类型 (POST)

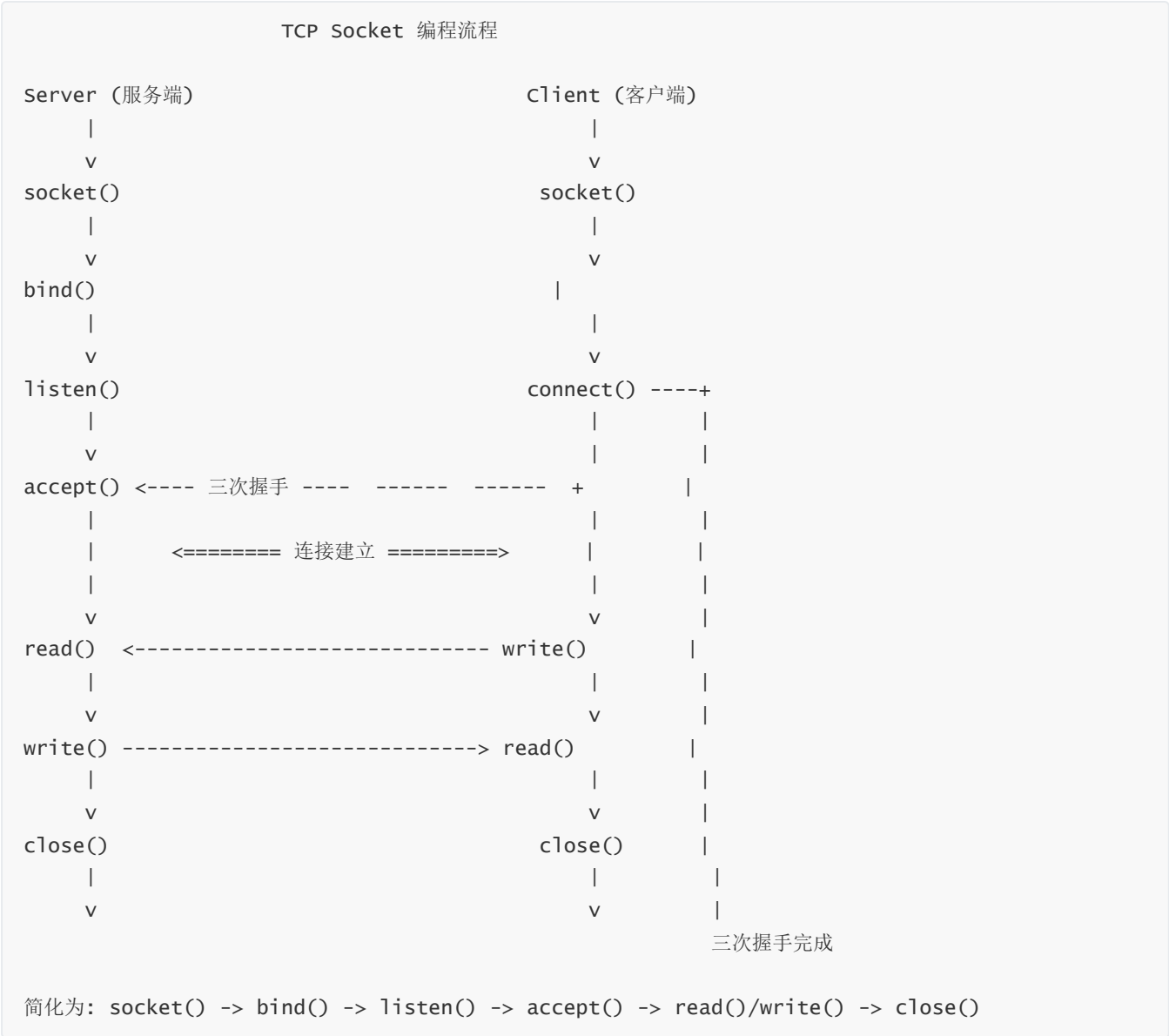
分类	头部	说明
响应	Content-Length	请求体长度
	Content-Type	响应体类型
	Content-Length	响应体长度
	Content-Encoding	压缩编码 (gzip, br)
	Set-Cookie	设置 Cookie
	Cache-Control	缓存策略
	Location	重定向目标 (3xx)
	Access-Control-Allow-Origin	CORS 跨域
通用	Connection	keep-alive / close
	Transfer-Encoding	chunked (分块传输)
	Date	消息创建时间

7.5 RESTful API 设计

原则	说明	示例
资源 (Resource)	将系统中的实体抽象为资源，用 URL 标识	/users/123, /orders/456
HTTP 方法 (Verb)	使用 HTTP 方法表示操作	GET(查), POST(增), PUT(改全), PATCH(改部), DELETE(删)
无状态	每个请求包含所有需要的信息	使用 Token 而非 Session
表现层	资源可以有多种表示	Accept: application/json 或 Accept: application/xml
HATEOAS	响应包含相关链接 (超媒体)	{"links": {"orders": "/users/123/orders"}}
状态码	使用标准 HTTP 状态码	200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error

8. Socket 基础

8.1 Socket 编程模型



8.2 Socket 关键函数

```
// 创建 socket
int socket(int domain, int type, int protocol);
// domain: AF_INET (IPv4), AF_INET6 (IPv6), AF_UNIX (Unix域)
// type: SOCK_STREAM (TCP), SOCK_DGRAM (UDP), SOCK_RAW (原始套接字)
// protocol: 0 (自动), IPPROTO_TCP, IPPROTO_UDP

// 绑定地址
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

// 监听 (TCP服务端)
int listen(int sockfd, int backlog);
// backlog: 全连接队列 (accept queue) 的最大长度
```

```
// 接受连接 (TCP服务端)
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
// 返回新的 fd 用于通信

// 发起连接 (TCP客户端)
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

// 关闭
int close(int sockfd);
int shutdown(int sockfd, int how);
// how: SHUT_RD (关读), SHUT_WR (关写), SHUT_RDWR (关读写)
```

面试题 19: listen 的 backlog 参数到底是什么?

答: `listen(fd, backlog)` 中的 `backlog` 指定了**全连接队列 (已完成三次握手的连接)** 的最大长度。注意:

- 不是半连接队列 (SYN queue) 的大小 (那个由 `tcp_max_syn_backlog` 控制)
- 全连接队列中的连接等待 `accept()` 取走
- 如果全连接队列满了, 新的连接请求:
 - `/proc/sys/net/ipv4/tcp_abort_on_overflow = 1`: 发送 RST
 - `/proc/sys/net/ipv4/tcp_abort_on_overflow = 0` (默认): 忽略 SYN+ACK 的 ACK, 让客户端重传, 期望队列很快有空位

面试重点

高频考点

1. **三次握手/四次挥手** — 必考! 要能画出时序图, 写出每次的状态、序列号、ACK值的变化。
2. **TIME_WAIT** — 为什么需要 (两大原因)、2MSL、端口耗尽如何处理。
3. **拥塞控制** — 慢启动、拥塞避免、快速重传、快速恢复四个阶段, 能画出 `cwnd` 锯齿图。
4. **TCP 和 UDP 区别** — 不只是"可靠 vs 不可靠", 要从连接、边界、头部开销、适用场景等多维度对比。
5. **粘包拆包** — 产生原因, 解决方案 (重点: 长度前缀)。
6. **滑动窗口和流量控制** — 窗口机制, `rwnd` vs `cwnd`。
7. **HTTP/HTTPS 握手流程** — TLS 1.2/1.3 的握手流程, HTTP 各版本区别。
8. **DNS 解析流程** — 递归 vs 迭代查询, 各级缓存。
9. **TCP 状态转换** — `CLOSE_WAIT` 产生原因, 排查方法。

常见陷阱

1. **三次握手 "第三次ACK丢失" 的误解** — 客户端认为连接已建立, 服务器重传 SYN+ACK。如果客户端不发数据, 这个连接在服务器端可能一直保持在 `SYN_RCVD` 状态直到超时。
2. **四次挥手 "只看到三次"** — 如果被动关闭方在收到 FIN 后没有数据要发, ACK 和 FIN 可以合并在一个报文中, 出现"三次挥手"。
3. **TCP 是可靠协议, 所以数据不会丢** — 错! TCP 可靠性靠重传保证, 丢包仍然会发生在网络层, TCP 只是保证最终收到。

4. **Nagle 算法总是好的** — 在低延迟交互场景，Nagle 会导致不可接受的延迟，需要 TCP_NODELAY。
5. **SO_REUSEADDR 可以解决所有 TIME_WAIT 问题** — 它只解决 bind 失败问题 (服务端重启)，不解决客户端端口耗尽问题。
6. **HTTP/1.1 的 Keep-Alive 不会造成队头阻塞** — 会! HTTP/1.1 的连接上请求必须按序处理，前面的请求阻塞了后面的 (HTTP 层面的 Head-of-Line Blocking)。
7. **拥塞窗口 (cwnd) 一定等于发送窗口** — 不对，实际发送窗口 = $\min(\text{cwnd}, \text{rwnd})$ ，取两者的最小值。

建议背诵内容

必须能脱稿画出的图:

- TCP 头部格式 (每个字段的名称和位置)
- 三次握手时序图 (含状态转换和序列号)
- 四次挥手时序图 (含状态转换和 TIME_WAIT)
- 数据的封装过程 (应用 -> 传输 -> 网络 -> 链路)
- TCP Reno 拥塞控制状态机
- DNS 解析流程图
- TLS 1.2 握手流程图
- OSI 7层和 TCP/IP 4层对比

必须能脱稿回答的问题:

1. 为什么三次握手? 为什么不是两次或四次? (面试题3)
2. 为什么需要 TIME_WAIT? 持续多久? 太多怎么办? (面试题6)
3. TCP 拥塞控制的四个算法分别做了什么? 画出 cwnd 变化图。(面试题10, 11)
4. TCP 粘包是什么? 如何解决? (面试题14)
5. 浏览器输入 URL 后发生了什么? (综合: DNS -> TCP -> HTTP -> 渲染)
6. CLOSE_WAIT 状态是什么? 大量 CLOSE_WAIT 怎么排查? (面试题7)
7. TCP 和 UDP 的区别? 分别适用什么场景? (面试题15)
8. Nagle 算法是什么? 什么时候禁用它? (面试题13)
9. HTTP/1.1 vs HTTP/2 vs HTTP/3 的区别?
10. HTTPS 握手流程? TLS 1.2 vs 1.3 的区别?

9. 深入专题

9.1 TCP KeepAlive 机制

TCP KeepAlive 用于检测空闲连接的对端是否仍然存活。

KeepAlive 机制:

TCP连接建立后，如果长时间没有数据传输，KeepAlive 会:

`tcp_keepalive_time` (7200s = 2小时) 空闲后，开始探测

每 `tcp_keepalive_intvl` (75s) 发送一个探测包

连续 `tcp_keepalive_probes` (9次) 没有响应

-> 判定对端已经不可达, 关闭连接

KeepAlive 探测报文:

- 是一个空的 ACK 报文 (序列号是对方期望的 -1, 对方会回复 ACK)
- 或者带有 1 字节垃圾数据的报文

应用层 KeepAlive vs TCP KeepAlive:

应用层: 如 HTTP 的心跳请求, Redis 的 PING-PONG

TCP层: 传输层自动检测, 对应用透明

设置 (Socket 选项):

`SO_KEEPALIVE` - 开启

`TCP_KEEPIDLE` - 空闲时间

`TCP_KEEPINTVL` - 探测间隔

`TCP_KEEPCNT` - 探测次数

面试题: TCP KeepAlive 和 HTTP Keep-Alive 有什么区别?

答:

- TCP KeepAlive: 传输层机制, 用于检测**对端进程是否存活**。长时间无数据后发探测包确认连接是否有效。
- HTTP Keep-Alive: 应用层机制 (HTTP/1.1 默认), 用于**复用 TCP 连接**发送多个 HTTP 请求/响应, 减少 TCP 连接建立开销。即使有 HTTP Keep-Alive, TCP 连接上仍有 keepalive 机制在监测连接健康。

9.2 TCP 的 11 种状态全览

TCP 11种连接状态完整梳理:

LISTEN	- 服务端, 等待客户端连接请求 (SYN)
SYN_SENT	- 客户端, 已发送 SYN, 等待 SYN+ACK
SYN_RCVD	- 服务端, 收到 SYN, 发回 SYN+ACK, 等待 ACK
ESTABLISHED	- 连接已建立, 正常数据传输
FIN_WAIT_1	- 主动关闭, 已发送 FIN, 等待 ACK 或对方的 FIN
FIN_WAIT_2	- 主动关闭, 已收到 ACK, 等待对方的 FIN
CLOSING	- 双方同时关闭, 已发 FIN, 收到 FIN 但未收到自己的 FIN 的 ACK
TIME_WAIT	- 主动关闭, 发了最后一个 ACK, 等待 2MSL
CLOSE_WAIT	- 被动关闭, 收到 FIN, 已发 ACK, 等待应用层 close()
LAST_ACK	- 被动关闭, 已发 FIN, 等待最后一个 ACK
CLOSED	- 无连接

状态迁移特殊情况:

同时打开 (Simultaneous Open):

Client: CLOSED -> SYN_SENT (收SYN前收到对端SYN) -> SYN_RCVD -> ESTABLISHED

Server: 类似

(双方都收到了对方的 SYN, 交换了 4 个报文)

同时关闭 (Simultaneous close):

双方同时发 FIN, 进入 FIN_WAIT_1

收到对端 FIN (而非 ACK) -> 进入 CLOSING

收到 ACK -> 进入 TIME_WAIT

关于 CLOSING 状态：

这是很少见的状态，条件是：

1. 我方发了 FIN (进入 FIN_WAIT_1)
 2. 还没收到我方的 FIN 的 ACK
 3. 先收到了对方的 FIN
- > 我方回复 ACK + 进入 CLOSING
- > 收到我方的 FIN 的 ACK -> TIME_WAIT

9.3 TCP 定时器

TCP 的 4 种主要定时器：

1. 重传定时器 (Retransmission Timer)

- 每个发送的报文段设置一个定时器
- 超时未收到 ACK，重传该报文
- 超时时间 RTO (Retransmission Timeout)
- RTO 基于 RTT 动态计算 (Jacobson/Karels 算法)
- 超时后 RTO 翻倍 (指数退避)

SRTT (Smoothed RTT):

$$SRTT = (1-\alpha) * SRTT + \alpha * RTT_sample \quad (\alpha = 1/8)$$

RTTVAR (RTT Variation):

$$RTTVAR = (1-\beta) * RTTVAR + \beta * |SRTT - RTT_sample| \quad (\beta = 1/4)$$

$$RTO = SRTT + 4 * RTTVAR$$

2. 持续定时器 (Persist Timer)

- 解决零窗口死锁
- 接收方通告 win=0 -> 发送方停止发送
- 接收方发送窗口更新报文 (win>0)，但此报文丢失！
- 双方互相等待 -> 死锁
- 持续定时器：发送方收到 win=0 后启动，定时发送零窗口探测

3. 保活定时器 (Keepalive Timer)

- 见 9.1 节

4. TIME_WAIT 定时器

- 进入 TIME_WAIT 后启动
- 2MSL 后超时，进入 CLOSED

重传超时 vs 快速重传：

超时重传：定时器触发，慢 (需要等 RTO，通常 200ms+)

快速重传：3 DupACK 触发，快 (通常在 1-2 RTT 内)

面试题: RTO 如何计算? 为什么需要 RTTVAR?

答:

- $RTO = SRTT + 4 * RTTVAR$
- 不能只用 SRTT，因为 RTT 波动很大。如果 RTT 突然变大 (网络拥塞)，用单纯的 SRTT 作为 RTO 会过早超时，导致不必要的重传。

- Jacobson 算法引入了 RTTVAR (RTT 变化的平滑平均值), 将变异纳入超时计算。
- 如果超时重传, 使用 Karn 算法: 重传的报文收到 ACK 时, 不知道该 ACK 是对原报文的确认还是对重传报文的确认, 因此该 RTT 样本不用于更新 SRTT。RTO 翻倍 (指数退避)。

9.4 TCP 选项详解

TCP 时间戳选项 (Timestamps Option, RFC 1323):

```
+-----+-----+-----+-----+
| Kind  | Length| TSval (4 bytes) | TSecr (4 bytes) |
| = 8   | = 10  | 发送方当前时间戳 | 回显对端的时间戳 |
+-----+-----+-----+-----+
```

两大作用:

1. RTTM (Round Trip Time Measurement):

更精确的 RTT 估计 (不依赖 1 RTT 一个样本的限制)

通过 TSval 和 TSecr 计算每个 ACK 对应的 RTT

2. PAWS (Protection Against Wrapped Sequences):

在高带宽网络 (如 10Gbps+) 中, 32位序列号可能很快回绕

(TCP 序列号空间: $2^{32} = 4GB$, 在 10Gbps 下 3.4 秒就回绕!)

时间戳 (32位, 通常 1ms 或更短精度) 比序列号更慢回绕

如果收到的报文的 TSval < 上次记录的 TSval, 认为是旧报文, 丢弃

SACK (Selective Acknowledgment) 选项:

问题: 累积 ACK 只告诉发送方 "期望的下一个序列号"

如果多个报文段丢失, 发送方不知道哪些丢了

SACK: 告诉发送方接收方已经收到了哪些不连续的数据块

SACK 选项格式:

```
+-----+-----+-----+-----+
| Kind  | Length| Left Edge 1    | Right Edge 1    |
| = 5   |       | (第一个块的起始) | (第一个块的结束+1) |
+-----+-----+-----+-----+
(支持最多 4 个块, 在 TCP 选项 40 字节空间允许范围内)
```

SACK 的好处:

- 发送方可以只重传丢失的报文段 (而不是整个窗口)
- 减少不必要的重传
- 在网络有多个丢包时效果显著

window scale (窗口扩大因子) 选项:

问题: TCP 头部 window 字段只有 16 位, 最大窗口 = 65535 字节

在高带宽长延迟 (BDP 大) 网络中, 65535 根本不够!

BDP (Bandwidth-Delay Product):

带宽 10Gbps, RTT 100ms $\rightarrow$ $BDP = 10Gbps * 0.1s = 1Gbit = 125MB$

需要 125MB 的窗口才能填满管道, 而 65535 字节只有 0.064MB!

window Scale: 窗口值 = 头部window * 2^{shift}
shift 范围 0-14, 所以最大窗口 = $65535 * 2^{14} \approx 1\text{GB}$

9.5 网络性能调优

Linux 网络栈参数调优

```
# === TCP 缓冲区调优 ===
# 接收缓冲区 (自动调优范围)
net.ipv4.tcp_rmem = "4096 131072 6291456"
#               min    default  max (单个连接)
# 发送缓冲区
net.ipv4.tcp_wmem = "4096 65536 4194304"

# 缓冲区内内存使用
net.core.rmem_max = 16777216 # 最大接收缓冲区 (所有协议)
net.core.wmem_max = 16777216 # 最大发送缓冲区
net.core.rmem_default = 212992
net.core.wmem_default = 212992

# TCP 内存 (全局, 单位: pages)
net.ipv4.tcp_mem = "786432 1048576 1572864"
#               min    pressure  max (超过pressure进入内存压力模式)

# === 连接队列调优 ===
net.core.somaxconn = 4096 # accept 队列上限
net.ipv4.tcp_max_syn_backlog = 8192 # SYN 队列上限 (半连接队列)
net.core.netdev_max_backlog = 5000 # 网卡 backlog (软中断来不及处理的包)

# === FIN/KeepAlive 调优 ===
net.ipv4.tcp_fin_timeout = 30 # FIN_WAIT_2 -> TIME_WAIT 超时
net.ipv4.tcp_tw_reuse = 1 # 允许复用 TIME_WAIT (客户端)
net.ipv4.tcp_max_tw_buckets = 5000 # 最大 TIME_WAIT 数量
net.ipv4.tcp_keepalive_time = 7200 # KeepAlive 空闲时间
net.ipv4.tcp_keepalive_intvl = 75 # KeepAlive 探测间隔
net.ipv4.tcp_keepalive_probes = 9 # KeepAlive 探测重试次数

# === 性能特性 ===
net.ipv4.tcp_sack = 1 # 启用 SACK
net.ipv4.tcp_timestamps = 1 # 启用时间戳 (RTTM + PAWS)
net.ipv4.tcp_window_scaling = 1 # 启用窗口扩大因子
net.ipv4.tcp_fastopen = 3 # 启用 TCP Fast Open (客户端+服务端)
net.ipv4.tcp_syncookies = 1 # 启用 SYN Cookie 防 SYN Flood
net.ipv4.tcp_mtu_probing = 1 # 启用 PMTU 探测

# === 多队列/多核 ===
net.core.rps_sock_flow_entries = 32768 # RPS (Receive Packet Steering)
```

```
# 每个网卡的 RPS CPU 掩码在 /sys/class/net/<iface>/queues/ 中设置
# RFS (Receive Flow Steering) 加速
```

TCP Fast Open (TFO)

TCP Fast Open (RFC 7413):

问题：三次握手完成前不能传输数据，1 RTT 的延迟

TFO 解决：

1. 客户端第一次连接时（普通三次握手）：
服务端在 SYN+ACK 中携带一个 Cookie
2. 客户端第二次及以后连接时：
在 SYN 报文中携带 Cookie + 应用数据
服务端验证 Cookie 有效 -> 立即交付数据！
3. 相比传统：节省 1 RTT 延迟

局限性：

- 需要双端支持（客户端 + 服务端）
- Cookie 的安全性（需要定期更换密钥）
- SYN 中可携带的数据有限（MSS 限制）

9.6 ICMP 协议

ICMP (Internet Control Message Protocol):

封装在 IP 数据报中 (Protocol = 1)

常见类型：

- Type 0: Echo Reply (ping 应答)
- Type 3: Destination Unreachable (目标不可达)
 - Code 0: 网络不可达
 - Code 1: 主机不可达
 - Code 3: 端口不可达 (UDP, 无监听进程)
 - Code 4: 需要分片但设置了 DF 标志 (PMTUD 用)
- Type 8: Echo Request (ping 请求)
- Type 11: Time Exceeded (TTL=0, traceroute 用)
- Type 5: Redirect (重定向, 更好的路由)

Traceroute 原理：

发送 TTL=1, 2, 3, ... 的 UDP 包

每经过一个路由器 TTL 减1

当 TTL=0 时路由器丢弃并返回 ICMP Time Exceeded

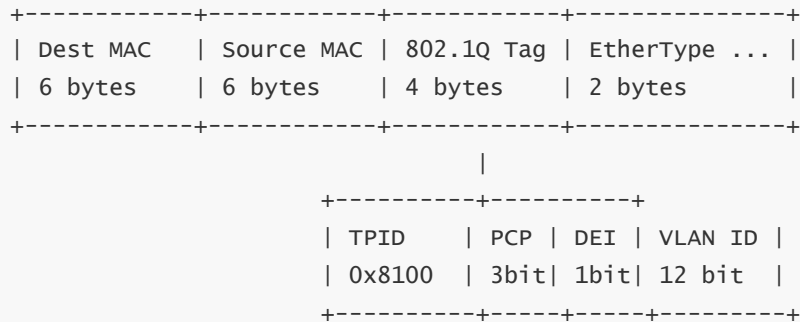
最终到达目标返回 ICMP Port Unreachable (用了一个未开放的端口)

-> 获得到达路径上每个路由器的信息

9.7 VLAN 与 交换机

VLAN (Virtual LAN, 802.1Q):

在以太网帧中插入 4 字节的 802.1Q 标签:



作用:

- 在同一个物理交换机上隔离不同广播域
- VLAN ID (1-4094) 标识虚拟网络
- Trunk 端口: 携带多个 VLAN 的流量 (通过 tag 区分)
- Access 端口: 只属于一个 VLAN (不打 tag 或去除 tag)

9.8 NAT (Network Address Translation)

NAT 类型:

1. 静态 NAT: 一对一映射 (内部 IP <-> 外部 IP)
2. 动态 NAT: 池内动态映射
3. PAT/NAPT (端口多路复用): 最常用, 多个内部 IP 共享一个公网 IP (通过不同端口来区分)

NAT 工作流程:

内网主机 (192.168.1.10:12345) -> 百度 (110.242.68.66:80)

发送方 (内网):

src=192.168.1.10:12345, dst=110.242.68.66:80

经过 NAT 路由器:

src=1.2.3.4:56789 (公网IP:新端口), dst=110.242.68.66:80

并在 NAT 表中记录映射:

192.168.1.10:12345 <-> 1.2.3.4:56789

百度回复:

src=110.242.68.66:80, dst=1.2.3.4:56789

NAT 路由器查表:

1.2.3.4:56789 -> 192.168.1.10:12345

转发: src=110.242.68.66:80, dst=192.168.1.10:12345

NAT 的问题:

- 破坏了端到端原则 (IP 地址被修改)
- 内网主机无法被外网主动连接 (需要端口映射/UPnP)
- NAT 表项有超时 (UDP "连接" 超时后映射消失)
- 增加路由器 CPU 开销

9.9 综合场景: 从输入 URL 到页面显示

这是经典的"全栈"面试题, 需要串联 DNS, TCP, TLS, HTTP, 浏览器渲染。

完整流程 (假设输入 `https://www.example.com/path`):

1. 键盘/输入法处理
 - 操作系统处理键盘中断
 - 输入法 (IME) 处理
2. 解析 URL
 - scheme: `https` (端口 443)
 - host: `www.example.com`
 - path: `/path`
3. DNS 解析 (详见第6节)
 - 浏览器缓存 -> OS缓存 -> hosts -> LDNS -> 递归/迭代查询
 - 得到 IP: `93.184.216.34`
4. 建立 TCP 连接 (三次握手, 详见2.2节)
 - Client: SYN (seq=x)
 - Server: SYN+ACK (seq=y, ack=x+1)
 - Client: ACK (seq=x+1, ack=y+1)
 - 连接建立, RTT 耗时
5. TLS 握手 (HTTPS, 详见7.2/7.3节)
 - TLS 1.2: 2-RTT (ClientHello, ServerHello+Cert, ClientKEX+Finished, Finished)
 - TLS 1.3: 1-RTT
 - 协商密码套件, 交换密钥, 建立加密通道
6. 发送 HTTP 请求
 - GET /path HTTP/1.1
 - Host: `www.example.com`
 - User-Agent, Accept, Cookie, ...
7. 服务端处理
 - 反向代理 (Nginx): SSL 终止, 负载均衡
 - 应用服务器: 路由 -> 中间件 -> 控制器 -> 业务逻辑
 - 可能涉及: 缓存 (Redis), 数据库 (MySQL), 消息队列
8. HTTP 响应
 - HTTP/1.1 200 OK
 - Content-Type: `text/html; charset=utf-8`
 - Content-Encoding: `gzip`
 - Set-Cookie, Cache-Control, ...
 - HTML 内容

9. 浏览器解析 HTML

- DOM 树构建
- 遇到外部资源 (CSS, JS, 图片):
 - 可能会创建新的 TCP 连接 (或复用已有连接)
 - 并行下载 (HTTP/1.1 通常 6 个并发连接/域名)
- CSS: CSSOM 树构建 (阻塞渲染)
- JS: 执行 (阻塞 DOM 解析, 除非 async/defer)

10. 渲染

- Render Tree (DOM + CSSOM 合并)
- Layout (布局/回流): 计算每个元素的位置和大小
- Paint (绘制): 将元素绘制到图层
- Composite (合成): 将图层合并显示到屏幕

11. 交互

- JavaScript 事件绑定
- AJAX / WebSocket 后续异步请求
- 页面可交互

全流程耗时分解 (典型值):

DNS 解析: 20-120ms

TCP 握手: 1 RTT (取决于距离, 同城 10ms, 跨国 200ms+)

TLS 握手: 1-2 RTT

HTTP 请求: 1 RTT+

服务端处理: 取决于服务端性能

内容下载: 取决于带宽和内容大小

浏览器渲染: 取决于页面复杂度

总耗时 (首次访问): 通常 500ms - 3s

10. 高频面试题汇总

TCP 专项面试题

Q1: TCP 如何保证可靠性?

答:

1. **序列号+确认**: 每个字节编号, 接收方确认已收到
2. **超时重传**: 发送方启动定时器, 超时未收到ACK则重传
3. **校验和**: 检测传输过程中的数据损坏
4. **流量控制**: 接收方通过窗口大小控制发送速率
5. **拥塞控制**: 发送方根据网络状况调整发送速率
6. **连接管理**: 三次握手+四次挥手
7. **数据排序**: 接收方按序列号重新排序

Q2: TCP 的缺陷是什么?

答:

1. 队头阻塞 (Head-of-Line Blocking): TCP 层面的丢包导致后续所有包阻塞
2. 连接建立延迟: 三次握手 + TLS 握手, 首次连接需要多趟 RTT
3. 连接迁移: 切换网络 (Wi-Fi -> 4G) 时 TCP 连接断开, 需要重建
4. 握手开销: 短连接场景下握手/挥手开销大
5. 不适合无线/高丢包网络: 拥塞控制误判丢包原因 (拥塞 vs 无线干扰)

这些缺陷正是 QUIC (HTTP/3) 试图解决的问题。

Q3: 如何设计一个可靠的 UDP 传输协议?

答 (参考 QUIC/KCP):

1. 序列号 + ACK 确认 (可靠传输)
2. 重传机制 (超时 + NACK)
3. 前向纠错 (FEC): 冗余编码, 丢包可从冗余恢复
4. 流量控制: 接收窗口
5. 拥塞控制: 自定义策略 (如 BBR 风格)
6. 连接管理: Connection ID (支持连接迁移)

Q4: TCP 序列号回绕问题是什么?

答: TCP 序列号是 32 位 ($2^{32} = 4\text{GB}$)。在高带宽网络中:

- 10Gbps: $4\text{GB} / (10\text{Gbps}/8) = 3.2$ 秒回绕
- 100Gbps: 0.32 秒回绕

回绕后无法区分新旧报文。解决方案:

- PAWS (Protection Against Wrapped Sequences): 使用 TCP 时间戳选项
- 如果时间戳增长比序列号回绕慢, 可以区分新旧

Q5: TCP 粘包和 UDP 粘包有什么区别?

答: UDP 不会粘包! UDP 保留消息边界, 一次 sendto 对应一次 recvfrom。但需要注意:

- 如果 UDP 数据报 > MTU, 会在 IP 层分片, 接收方 IP 层重组。应用层仍是一次 recvfrom 得到完整包。
- UDP 缓冲区满时, 新的数据报会被丢弃 (不是截断)。
- 应用层 UDP 协议 (如 DNS) 本身可能丢包, 需要应用层超时重传。

Q6: 什么是 Silly Window Syndrome (傻瓜窗口综合症)?

答: 两种场景:

1. 接收方应用每次只读 1 字节, 窗口每次只开放 1 字节 → 发送方发送大量 1 字节报文
2. 发送方发送数据很慢, 每次只够填充 1 字节 MSS, 导致大量小包

解决方案:

- 接收方: Clark 算法 — 窗口小于一定值 (或缓冲区的一半) 时声明 0 窗口, 等应用多读一些
- 发送方: Nagle 算法 — 限制小包的发送 (见2.9节)

HTTP/HTTPS 专项面试题

Q7: HTTP/2 的队头阻塞问题解决了吗?

答: 部分解决。HTTP/2 在单个 TCP 连接上实现了**多路复用 (Multiplexing)**，解决了 HTTP/1.x 的 HTTP 层面的队头阻塞 (请求-响应的串行处理)。但是 TCP 层面的队头阻塞仍然存在: 如果一个 TCP 报文段丢失，整个 TCP 连接上的所有流都被阻塞 (虽然只有一条流的数据丢失)。HTTP/3 (基于 QUIC/UDP) 彻底解决了这个问题。

Q8: HTTPS 一定能保证安全吗?

答: HTTPS (TLS + HTTP) 提供了机密性、完整性、身份认证。但是:

- 1. TLS 版本过旧 (SSL 3.0, TLS 1.0) 有已知漏洞
- 2. 证书验证可能被绕过 (中间人攻击)
- 3. 证书机构 (CA) 被攻破可能签发伪造证书
- 4. 私有密钥泄露
- 5. 降级攻击 (强制使用弱密码套件)
- 6. 应用层漏洞 (XSS, CSRF) 不受 TLS 保护

Q9: 为什么 HTTPS 比 HTTP 慢?

答:

- 1. TLS 握手: 额外的 1-2 RTT (TLS 1.2 需要 2-RTT + TCP 的 1-RTT = 3-RTT)
- 2. 加密/解密: CPU 开销 (现代 CPU 有 AES-NI 硬件加速, 影响变小)
- 3. 证书链传输: 服务器需要发送证书链 (通常 3-5KB)
- 4. 没有缓存时: 每次握手都需要计算密钥

优化:

- TLS 1.3: 握手减为 1-RTT
- Session Resumption: 复用之前的会话密钥 (0-RTT)
- OCSP Stapling: 服务器附带已签名的证书状态
- HTTP/2: 多路复用减少连接数

11. Linux 网络子系统深入

11.1 Socket Buffer (sk_buff / skb)

`sk_buff` 是 Linux 内核中表示网络数据包的核心数据结构。每个网络包 (从网卡接收到发送到应用层) 在内核中都由一个 `skb` 表示。

sk_buff 结构示意:

```
+-----+
| struct sk_buff                                     |
+-----+
| *next, *prev          - 链表节点 (sk_buff_head)    |
| *sk                   - 所属 socket                |
| tstamp                - 时间戳                     |
```

```

| *dev          - 网络设备          |
|
| 四指针管理数据：                |
| transport_header -> +-----+      |
|                    | Ethernet Header (14 B) | |
| network_header  -> | IP Header (20 B)      | |
|                    | TCP Header (20 B)      | |
| mac_header      -> +-----+      |
|                    | ...                    | |
| head            -> +-----+      |
| data            -> | 有效载荷数据          | |
|                    |                      | |
| tail            -> +-----+      |
| end             -> | Linear Data (数据区)   | |
|                    +-----+      |
|
| len: 数据长度 (从 data 到 tail)    |
| data_len: 非线性的分片数据总长度  |
| truesize: skb 总共占用的内存 (含 skb_shared_info) |
|
| +-- skb_shared_info (共享信息区)   |
|     nr_frags: 分片数量              |
|     frags[]: 页面分片数组 (用于零拷贝) |
|     frag_list: 分片的 skb 链表      |
+-----+

```

使用 `skb` 的零拷贝：

- 网卡用 `DMA` 将数据包放入内核预分配的页面
- `skb->data` 指向这些页面
- `sendfile()` 将文件页面映射到 `skb` 的 `frags[]` (零拷贝)

11.2 Netfilter / iptables 框架

Netfilter 是 Linux 内核中的包过滤框架，iptables 是其用户空间管理工具 (新版本是 nftables)。

Netfilter 的 5 个钩子 (Hook) 点：



网卡收到数据包

网卡发出数据包

数据包流向：

入站：网卡 -> PREROUTING -> 路由（本机） -> INPUT -> 用户空间

转发：网卡 -> PREROUTING -> 路由（转发） -> FORWARD -> POSTROUTING -> 网卡

出站：用户空间 -> OUTPUT -> 路由 -> POSTROUTING -> 网卡

iptables 表和链：

raw: PREROUTING, OUTPUT (connection tracking 绕过)

mangle: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING (包修改)

nat: PREROUTING, INPUT, OUTPUT, POSTROUTING (NAT)

filter: INPUT, FORWARD, OUTPUT (过滤)

常见的 iptables 规则：

允许 80 端口

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

端口转发 (DNAT)

```
iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 10.0.0.1:80
```

出站 SNAT (MASQUERADE)

```
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE
```

限制连接速率

```
iptables -A INPUT -p tcp --dport 22 -m connlimit --connlimit-above 3 -j REJECT
```

```
iptables -A INPUT -p tcp --dport 22 -m limit --limit 3/min -j ACCEPT
```

11.3 网卡多队列与 RSS/RPS/RFS

网卡数据接收的多核优化：

RSS (Receive Side Scaling)：

- 硬件功能：网卡按 hash (src_ip, dst_ip, src_port, dst_port) 分配数据包到不同队列
- 每个队列有独立的 MSI-X 中断，绑定到不同 CPU
- 硬件做了第一次分流

RPS (Receive Packet Steering)：

- 软件功能：在单队列网卡上，软中断中根据 hash 将处理转发到远程 CPU
- 配置：/sys/class/net/<iface>/queues/rx-0/rps_cpus
- 弥补硬件不支持 RSS 的情况

RFS (Receive Flow Steering)：

- 进一步优化：将数据包发给正在运行对应应用层 socket 的 CPU
- 提高缓存亲和性（应用和网络处理在同一 CPU）
- 配置：/proc/sys/net/core/rps_sock_flow_entries
- 配合 /sys/class/net/<iface>/queues/rx-N/rps_flow_cnt

XDP (eXpress Data Path)：

- 在网络栈最前端（网卡驱动层）处理数据包
- 通过 eBPF 实现可编程的包处理
- 性能极高：可以 bypass 整个内核网络栈
- 应用：DDoS 防御 (XDP_DROP)，负载均衡，数据包采样

11.4 tcpdump / Wireshark 实战

=== tcpdump 常用命令 ===

基本语法

```
tcpdump -i eth0 -nn -vvv -s 0 -w capture.pcap
```

抓特定主机的流量

```
tcpdump -i eth0 host 192.168.1.100
```

抓特定端口

```
tcpdump -i eth0 port 80 or port 443
```

抓 TCP SYN 包

```
tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0'
```

只抓 TCP 三次握手

```
tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn'
```

或者

```
tcpdump -i eth0 'tcp[tcpflags] == tcp-syn'
```

抓特定连接的所有包 (源IP + 源端口 + 目的IP + 目的端口)

```
tcpdump -i eth0 'src host 10.0.0.1 and src port 12345 and dst host 10.0.0.2 and dst port 80'
```

显示绝对序列号

```
tcpdump -i eth0 -S
```

分析 TCP 连接详细状态

```
tcpdump -i eth0 -nn 'tcp' -v
```

=== Wireshark 技巧 ===

过滤器:

```
# tcp.port == 80          - 特定端口
```

```
# tcp.flags.syn == 1      - SYN 包
```

```
# tcp.flags.reset == 1    - RST 包
```

```
# tcp.analysis.retransmission - 重传包
```

```
# tcp.analysis.duplicate_ack - 重复 ACK
```

```
# tcp.analysis.zero_window - 零窗口通知
```

```
# tcp.analysis.lost_segment - 疑似丢包
```

```
# http.request.method == "GET" - HTTP GET 请求
```

Wireshark Statistics:

Statistics -> Flow Graph - TCP 时序图 (可视化握手/挥手)

Statistics -> IO Graph - 吞吐量图表

Analyze -> Expert Info - 自动分析网络问题

11.5 网络故障排查方法论

常见网络问题排查步骤：

1. 物理层检查

```
$ ethtool eth0          # 链路状态，速率，双工
$ ip -s link show eth0  # 错误统计
$ cat /proc/net/dev     # 网卡收发统计
```

2. 网络层检查

```
$ ping -c 10 target      # 基本连通性
$ traceroute target      # 路径探测
$ mtr target             # ping + traceroute 结合
$ ip route               # 路由表
```

3. 传输层检查

```
$ ss -antp              # TCP 连接状态
$ netstat -s            # TCP/UDP 统计
$ cat /proc/net/snmp    # 原始 SNMP 统计
$ cat /proc/net/netstat # 扩展统计 (含 ListenOverflows 等)
```

4. 应用层检查

```
$ curl -v http://target  # HTTP 请求诊断
$ openssl s_client -connect target:443 # TLS 检查
$ nslookup/dig target    # DNS 解析检查
```

5. 抓包分析

```
$ tcpdump -i any -w /tmp/debug.pcap
在 Wireshark 中分析：是否有重传、RST、零窗口等
```

常见问题模式：

- 连接超时：防火墙/安全组阻止，路由不可达
- 连接被重置 (RST)：端口未监听，防火墙主动拒绝
- 大量 TIME_WAIT：短连接频繁，见2.4节
- 大量 CLOSE_WAIT：应用层未 close，见2.5节
- 高延迟：跨地域/卫星链路，缓冲区膨胀 (Bufferbloat)
- 间歇丢包：网络拥塞，网卡/交换机故障，MTU 问题
- 连接数打满：ulimit 限制，端口耗尽，conntrack 表满

综合设计题

Q10: 设计一个 IM (即时通讯) 系统的网络层

答：

1. **传输协议**: TCP (长连接, 保证消息可靠到达) 或 QUIC (支持连接迁移)
2. **连接保活**: 应用层心跳 (如 30 秒 Ping-Pong), 防止 NAT 表项超时和连接断开检测
3. **消息协议**: 长度前缀 + Protobuf (高效二进制序列化)
4. **消息可靠投递**:
 - 发送方: 消息ID + ACK 确认

- 超时重传
 - 接收方去重 (通过消息ID)
5. **在线状态:** WebSocket/长连接断开 = 离线
6. **离线消息:** 消息存入离线队列 (Redis/数据库), 上线后推送
7. **连接管理:**
- 客户端: 自动重连 + 指数退避
 - 服务端: 连接数限制、流控
8. **安全:** TLS 加密、Token 认证
9. **扩展性:** 分布式架构, 连接服务 (Gateway) 与业务逻辑 (Logic) 分离

12. QUIC 协议速览 (HTTP/3 基础)

QUIC (Quick UDP Internet Connections) 是 Google 设计的传输层协议, 已被 IETF 标准化为 HTTP/3 的底层协议。

QUIC 解决的问题:

+-----+-----+	
TCP 的痛点	QUIC 的解决方案
+-----+-----+	
队头阻塞 (TCP层面)	基于 UDP 的多路复用, 流独立
连接建立慢 (1+1+1 RTT)	0-RTT (首次 1-RTT, 之后 0-RTT)
连接迁移困难 (4元组绑定)	Connection ID (与IP/端口解耦)
底层不可升级 (内核中)	用户空间实现, 快速迭代
不强制加密	内置 TLS 1.3, 必选加密
+-----+-----+	

QUIC 核心特性:

1. 多路复用 (无队头阻塞):
 - 多个 HTTP 请求在一条 QUIC 连接中并发
 - 每个流独立, 流 A 丢包不影响流 B (不同于 HTTP/2+TCP)
2. Connection ID:
 - 连接标识符与 IP/端口无关
 - 从 Wi-Fi 切换到 4G: 连接不中断!
3. 0-RTT 握手:
 - 如果客户端之前连接过服务端, 可以缓存密钥
 - 下次连接时, 第一个包就可以携带应用数据
4. 改进的丢包恢复:
 - 单调递增的包序号 (Packet Number)
 - 重传时使用新的包序号 (避免 RTT 歧义)
 - 更精确的 RTT 测量
5. 前向纠错 (FEC, 早期版本):
 - 通过冗余编码在一定丢包范围内直接恢复, 无需重传 (当前 IETF QUIC 主要依赖重传而非 FEC)

6. 用户空间实现:

- 不在内核中 (UDP 是内核的, QUIC 在用户空间)
- 快速演进 (不需要升级内核)

连接迁移示例:

手机在 Wi-Fi (192.168.1.5:12345) 看视频

-> 走出 Wi-Fi 范围, 切换到 4G (10.0.0.5:54321)

-> TCP: 4元组变了, 连接断开 (视频卡顿, 需要重连)

-> QUIC: Connection ID 不变, 连接无缝继续!

面试题: HTTP/3 为什么基于 UDP 而不是重新设计一个传输层协议?

答:

1. 兼容性: 现有的网络设备 (路由器, NAT, 防火墙) 认识 TCP 和 UDP。设计新的 IP Protocol 号对中间设备是"陌生人", 往往被直接丢弃。
2. 部署难度: 通过 UDP 可以快速铺开, 不需要更新网络设备固件。
3. 用户空间: 不需要修改操作系统内核, 在用户空间 (应用库) 就可以实现 QUIC。
4. 实践证明: UDP 的"不可靠"只是基础传输, QUIC 在应用层实现了需要的可靠性和拥塞控制。

13. SCTP 简介

SCTP (Stream Control Transmission Protocol, RFC 4960):

相比 TCP 的特性:

1. 多宿主 (Multi-Homing):
 - 一个连接可以绑定多个 IP 地址 (对于冗余网络路径)
 - TCP: 一个连接只有一个 IP 对
 - SCTP: 同时连接 eth0 和 eth1, 一个路径故障自动切换
2. 多流 (Multi-Streaming):
 - 一个 SCTP 关联中有多个逻辑流
 - 流之间独立交付, 一个流的阻塞不影响其他流
 - TCP: 单字节流, 丢包队头阻塞
 - SCTP: 解决了 TCP 的队头阻塞 (类似 QUIC 但没有完全普及)
3. 面向消息:
 - 保留消息边界 (类似 UDP)
 - TCP 需要应用层自己界定消息边界
4. 四次握手 (更安全):
 - 防止 SYN Flood 攻击
 - 客户端: INIT ->
 - 服务端: INIT-ACK + Cookie
 - 客户端: COOKIE-ECHO
 - 服务端: COOKIE-ACK

应用场景:

- 电信信令 (SS7 over IP, SIGTRAN)
- WebRTC 数据通道 (底层的 Data Channel 使用 SCTP over DTLS over ICE/UDP)

- 由于中间件设备（NAT/防火墙）支持有限，SCTP 未广泛普及在互联网

上一章: [第三章 Linux底层](#)

下一章: [第五章 Socket编程与IO多路复用](#) — epoll、select、Reactor 模式等。

综合面试题 (TCP/IP + Linux):

1. 从浏览器输入 URL 到页面显示, 经历了哪些步骤? (DNS + TCP + TLS + HTTP + 渲染) — 详见 9.9 节
2. 一个 read() 调用在网络中经历了什么? (VFS -> TCP -> IP -> 驱动 -> 网络, 然后反向)
3. 如何设计一个能支持百万并发连接的系统? (Socket + epoll + 线程模型优化, 见Ch5)